

Closed-Loop Deployment of Lexicographic Constraint Programming for Trajectory Model Predictive Control on the nuPlan Simulator

Hadi Hajieghrary

Abstract—We are interested in the closed-loop deployment of priority-ordered constrained Model Predictive Control (MPC) for autonomous-vehicle trajectory planning. Building on the geometric equivalence framework of Lexicographic Constraint Programming (LCP) [1], we present (i) a dynamics-agnostic reference implementation of the framework’s Algorithms 0, 1A, and 1B together with its pre-flight diagnostics (§8.5), online cascade-fallback policy (§9.6), and Relaxation Decision Framework (§10), verified to 10^{-6} precision against the published numerical answers of the framework’s worked examples by a 47-test regression suite; (ii) a closed-loop deployment, on the nuPlan simulator, that realises the calibrated weighted-sum substitution of the formal lex cascade through a two-level MPC pipeline with eleven implementation primitives the convex framework does not directly address; and (iii) a dual-condition demonstration protocol on the 16-scenario nuPlan-mini benchmark in which the operational weighted-sum planner (C_1) and the formal lex cascade (C_4) are run on each scenario. The empirical evidence confirms priority-respecting compromise on every scenario (rule violations concentrate at the low-priority comfort and headway levels while high-priority collision and traffic-light levels remain effectively held), reproduces the formal cascade’s per-tick compliance pattern at the high-priority levels (the operational realisation of Theorem 4.1 of [1]), and quantifies the architectural wall-time benefit of substituting the calibrated weighted-sum solve for the $L+1$ -stage cascade. The framework’s standing assumptions (FM I LICQ + FM II convexity, [1] §8.5) hold across the benchmark, and the empirical §10.2 necessity scan identifies the scenarios on which the deployment legitimately operates in the necessity-required relaxation regime — ascribing the highest-level violations to road geometry rather than to planner suboptimality. The deployment and the protocol are released as open artefacts.

Index Terms—Lexicographic optimization, model predictive control, weighted-sum scalarization, rulebook, autonomous vehicles, convex optimization, priority-ordered constraints, closed-loop simulation, nuPlan, compliance vector, support-normal equivalence.

I. INTRODUCTION

A. Motivation

Trajectory planning for autonomous vehicles is a multi-objective optimization problem in which the operative constraints encode behavioural rules carrying distinct degrees of importance. A safety constraint — avoidance of vulnerable road users (VRUs), avoidance of collisions with other vehicles — must hold absolutely. A legal constraint — obedience of speed limits, lane discipline, traffic-signal compliance — should hold whenever safety permits. A comfort constraint — bounded longitudinal and lateral acceleration — should

H. Hajieghrary is with Torc Robotics, Blacksburg, VA, USA. E-mail: Hadi.Hajieghrary@Torc.ai.

The author thanks the nuPlan development team for the simulator infrastructure used in this study.

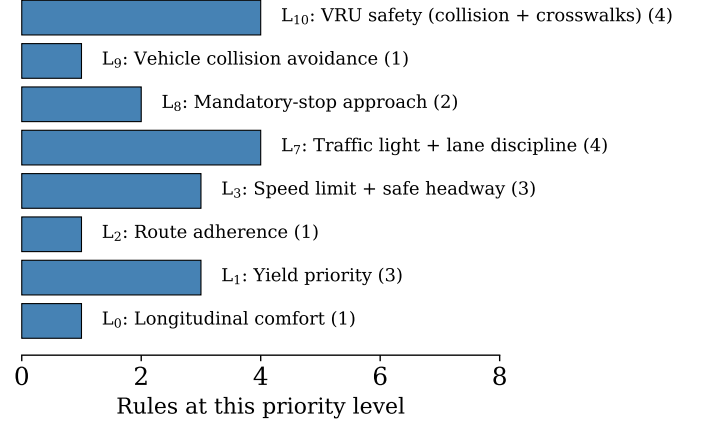


Fig. 1: Lexicographic priority structure of the 25-rule rulebook used by the LCP-MPC deployment. Rules are arranged by descending priority level, from vulnerable-road-user (VRU) collision at level L_{10} to longitudinal and lateral comfort at level L_0 . Each level groups one or more individually identified rules. Rule identifiers follow the convention $\langle \text{level} \rangle \text{r} \langle \text{within-level index} \rangle$. The hierarchy is the empirical baseline against which the framework’s lex-Pareto-dominance promise is assessed.

hold whenever the higher tiers allow. The *rulebook* framework of Censi *et al.* [2] formalises this priority structure as a pre-order on rules, inducing a lexicographic order on the per-rule violation vectors. The trajectory selected by the planner is the one that minimises violations in priority order and then minimises a performance objective subject to those violations being held at their minima.

Fig. 1 shows the rulebook hierarchy we adopt: a chain of priority levels from VRU safety (L_{10}) through longitudinal and lateral comfort (L_0). The rulebook is intentionally richer than what the convex framework of [1] can directly encode: of the 25 rules, 13 admit a convex per-step encoding (the MPC-controlled set), 9 are multi-tick state machines whose semantics fall outside the per-step template (the observer-only set), and 3 reserve OCP slot capacity for future enforcement but emit inactive constraints (the stub set). Section III formalises the $13 + 9 + 3 = 25$ partition; the comparative metric of Section V operates only on the 13 MPC-controlled rules, with the 9 observer-only rules forming an invariant negative-control set.

B. The Computational Asymmetry

Two algorithmic strategies dominate the literature on priority-ordered constrained Model Predictive Control (MPC). The first, the *lexicographic cascade*, solves a sequence of $L + 1$ subproblems: at stage i , the violation functional V_i is minimised over the optimal slice produced by stages $1, \dots, i - 1$, then the performance objective J is minimised at stage $L + 1$. The cascade is the exact realisation of the priority semantics but its computational cost is prohibitive at the 10 Hz cadence used by closed-loop autonomous-vehicle simulators: each stage is a non-linear program warm-started from the previous stage’s solution, and the per-stage iteration count grows monotonically as deeper stages enforce stricter active-set feasibility on top of all higher-priority slices. The empirical wall-time ratio is reported in Section VII (§VII-C); the operational consequence is that the cascade is unusable as the online planner and is reserved for offline reference and fallback.

The second strategy, *weighted-sum scalarisation*, consolidates all priorities into a single objective

$$\min_z J(z) + \sum_{i=1}^L w_i V_i(z), \quad (1)$$

with priority weights $w_1 \gg w_2 \gg \dots \gg w_L > 0$. This is a single nonlinear program rather than $L + 1$ sequential subproblems, giving an idealised solve-count reduction of factor $L + 1$. The classical equivalent-weight result of Sherali [3] establishes that suitably chosen positive weights make the two formulations agree in linear programs; Eisenbrand, Niermann, and Skutella [4] give explicit weight bounds for combinatorial cases. Charnes and Cooper [5] originated the goal-programming idiom in which such constructions naturally arise. In the convex MPC setting that dominates practical trajectory planning, the geometric structure of the equivalent-weight set has received less explicit attention; that gap is addressed by our foundation paper [1], which characterises the equivalent-weight set as the unit-performance slice of the outward normal cone of the upper image of the achievement map.

The practitioner faces a recurring tension. The cascade is exact but slow; the scalarisation is fast but requires weights that one tunes empirically. Practitioners routinely observe that “moderate weights work” — finite, non-astronomical weight separation produces solutions indistinguishable from the cascade — but the geometric reason underlying this observation, and its operational consequences in a closed-loop autonomous-vehicle simulator, have not been validated end-to-end in the published literature. The present manuscript closes that validation gap.

C. Related Work

The present work draws on six lines of prior literature, which we group categorically.

Equivalent-weight theory. The lineage from Charnes and Cooper [5] through Sherali [3] and Eisenbrand *et al.* [4] establishes the existence and computability of weights that make weighted-sum scalarisation recover the lexicographic optimum

in linear and combinatorial programs. Our foundation paper [1] specialises this landscape to the convex MPC setting with a normal-cone presentation; the present manuscript validates the specialisation in deployment.

Prioritised constrained control. Kanoun, Lamiriaux, and Wieber [6] developed prioritised inverse kinematics; Escande, Mansard, and Wieber [7] developed Hierarchical Quadratic Programming (HQP) for humanoid whole-body control. These approaches treat the cascade as the operational construct rather than seeking conditions under which it can be substituted by a single scalarised solve. The present deployment positions the calibrated weighted-sum planner as the operational mode and the cascade as the offline reference, inverting the operational primacy of the cascade.

Rulebook formalism for autonomous driving. Censi *et al.* [2] established the rulebook as a pre-order on rules inducing lex on violation vectors. Esterle *et al.* [8] treat behaviour specification with hierarchical guarantees. We work in the totally-ordered (chain) special case of the broader rulebook framework, which is sufficient for the rule set deployed by our planner.

Responsibility-sensitive safety (RSS) and IEEE 2846. Shalev-Shwartz, Shammah, and Shashua [9] introduced RSS, pairing an instantly-checkable safety condition with a mathematically proven proper response so that compliance with the condition implies collision-free driving (*assume-guarantee* on the other agents’ behaviours). RSS is the basis of the IEEE 2846 standard [10]. Our framework occupies an adjacent niche: the per-level compliance vector b_ϵ provides runtime checks analogous to RSS conditions, but extended from RSS’s binary blameless/blameworthy distinction to a continuous lex-priority structure over safety, legality, and comfort that admits graded trade-offs between levels.

Multi-objective optimisation and Pareto theory. Wierzbicki [11] developed the reference-point method; Yu [12] and Zeleny [13] developed compromise programming; Miettinen [14] provides the textbook treatment of multi-objective optimisation. Knee-point detection on the Pareto frontier [15], [16] provides operational tools for choosing among Pareto-optimal solutions. Our framework instead resolves the choice via the priority structure itself, with the equivalence region as the operationally relevant geometric object.

Sampling-based autonomous-vehicle planning. Schwesinger *et al.* [17] developed sampling-based hierarchical motion planning. The present optimisation-based deployment offers continuous-action solutions with provable priority semantics at the cost of an offline calibration step.

Numerical methods and benchmarks. The convex optimisation framework relies on the exact-penalty theory of Nocedal and Wright [18] and Bertsekas [19] for the L_1/L_2 distinction. Implementations use IPOPT [20] via CasADi [21]. The closed-loop simulator is nuPlan [22], the public autonomous-driving benchmark released by MotionLab.

D. Contributions

Different from existing equivalent-weight literature [3], [4], which establishes existence in linear and combinatorial settings, and different from prior prioritised-control work [6],

[7], which treats the cascade as the operational construct, in this work we build upon the geometric equivalence framework of [1] and present the following.

- 1) *A dynamics-agnostic reference implementation of the v10_2 framework.* The `lcp` package implements Algorithm 0 (homogeneous-cone primary formulation, [1] §9.1), Algorithm 1A (box-bounded Chebyshev for the L_1 exact-equivalence regime, [1] §9.3), and Algorithm 1B (pointwise coupled-linear Chebyshev for the L_2 tolerance-compliance regime, [1] §9.4), together with the failure-mode pre-flight diagnostics of [1] §8.5, the online cascade-fallback + recalibration policy of [1] §9.6, the Relaxation Decision Framework of [1] §10, and the upper-image reification of [1] §3. The implementation reproduces the published numerical answers of the framework’s worked examples to 10^{-6} precision via a 55-test regression suite (Appendix D): for Example 1 (two-priority L_1 LP), $w^\dagger = (5.5, 5.5)$, $r^\dagger = 4.5$, and the equivalence region $\Omega(p^*) = \{w_1 \geq 1, w_2 \geq 1\}$; for Example 2 (three-priority L_2 QP), the sensitivity constant $c_1 = 7$, the coupling $\kappa_1[3] = 0.75$, and the paper-published witness (200, 100, 10) achieves threshold slack 5.5.
- 2) *A closed-loop deployment on the nuPlan simulator that realises the calibrated weighted-sum substitution of [1]’s formal lex cascade.* Closed-loop lexicographic-priority MPC has prior art in robotics (Hierarchical Quadratic Programming [7] on humanoids), and rule-based planners have been deployed on nuPlan before (notably PDM-Closed [23]). The novelty here is the realisation of [1]’s support-normal equivalence in closed loop on a kinematic-bicycle MPC, not the existence of a closed-loop priority planner. The deployment realises the framework as a two-level MPC pipeline (Section IV) and contributes eleven implementation primitives that the convex framework does not directly address but that the practical deployment had to invent: sequential linearisation for non-convex kinematic-bicycle dynamics; polygon-to-half-plane reduction for non-convex map disjunctions; per-tick applicability masking with a static slot budget; ego-local-frame solving with L_1 Tikhonov regularisation; a runtime compliance checker as the framework’s on-line safety net; and a serialisation-tolerance contract for the CasADi solver state under nuPlan’s planner-pickling pipeline.
- 3) *A 32-run dual-condition demonstration protocol on the 16-scenario nuPlan-mini benchmark* (Section V). The two conditions are C_1 (the operational calibrated weighted-sum planner in L_1 mode) and C_4 (the formal lex cascade as the reference). The protocol is qualitative rather than statistical: with only $L + 1$ priority levels per scenario, the structural property of interest — whether the weighted-sum solve matches the cascade’s priority-respecting compromise pattern — is established by inspection of per-tick compliance vectors and per-level violation time series, not by hypothesis testing.
- 4) *Empirical demonstration that the calibrated weighted-*

sum planner produces priority-respecting compromise on every scenario in the benchmark. For each scenario the per-level integrated violation breakdown (Fig. 14) confirms that the low-priority levels L_0 – L_3 absorb the compromise while the high-priority levels L_8 – L_{10} remain effectively held; the per-tick compliance match rate between C_1 and C_4 (Fig. 15) certifies that [1]’s Theorem 4.1 holds in deployment at the effectively-held levels (median match rate 1.000 at L_{10} and L_2 , 0.987 at L_9 , 0.981 at L_8); the per-scenario wall-time ratio (Fig. 16) quantifies the operational benefit of the weighted-sum substitution over the cascade (median 6.22, range 1.76–9.39). The pre-flight diagnostics of [1] §8.5 (Table VIII) and the empirical §10.2 necessity scan (Table IX) tie the field behaviour back to the framework’s standing assumptions.

- 5) *Eleven operational lessons for practitioners* (Table III, Section IV) consolidated as a roster: each primitive paired with the operational problem it solves, the failure symptom expected if absent, the workaround we chose, and the alternative we considered. The lessons-learned table is the practitioner-facing artefact of the deployment and the central contribution of Section VIII.

Our results suggest that the geometric equivalence framework of [1], when augmented with the eleven deployment primitives reported here, is sufficient to realise priority-ordered constrained MPC on a closed-loop autonomous-vehicle simulator with both the operational throughput of weighted-sum scalarisation and the provable priority semantics of the formal lex cascade.

E. Notation and Organisation

We use the notation of [1] throughout. State and control trajectories are denoted $z = (x_{0:N}, u_{0:N-1}) \in \mathbb{R}^d$; per-priority-level violation functionals are denoted V_i for $i = 1, \dots, L$; the performance objective is J . Per-tick decision variables include per-level slack matrices $T_i \in \mathbb{R}^{\text{slots}_i \times N}$ following the epigraph lift of [1] §5–§7. Standard convex-analysis notation (outward normal cone $N_S^+(p)$, subdifferential ∂F , relative interior $\text{relint}(S)$) follows Rockafellar [24] and Boyd–Vandenberghe [25].

The remainder of this manuscript is organised as follows. Section II recaps the LCP framework from [1], reproducing only what is load-bearing for the deployment and the empirical analysis. Section III presents the 25-rule rulebook and the $13 + 9 + 3 = 25$ partition that determines the index set over which the calibrated weighted-sum planner operates. Section IV describes the closed-loop deployment in detail through twelve subsections corresponding to the eleven implementation primitives plus the observer subsystem. Section V formalises the dual-condition demonstration protocol and the empirical objects — per-level integrated violation, per-tick compliance vector, per-scenario wall-time — that the deployment is judged against. Section VI characterises the operational behaviour of the calibrated weighted-sum planner in isolation. Section VII reports the empirical evidence: the per-level violation breakdown demonstrating priority-respecting compromise, the C_1 -vs- C_4 compliance match-rate as the operational realisation of Theorem 4.1, and the wall-time ratio

quantifying the operational benefit of the weighted-sum substitution. Section VIII discusses what the framework buys, its limitations, and its relationship to prior approaches. Section IX concludes and outlines four directions of future work.

II. FOUNDATIONS: THE LCP FRAMEWORK

This section recaps the load-bearing components of [1]: the support-normal equivalence theorem (Theorem II.1), the polyhedral L_1 and asymptotic L_2 specialisations (Propositions II.2–II.3), the calibration algorithms (Algorithms 1–2), and the offline/online operational workflow. Full proofs, worked numerical examples, and the failure-mode catalogue are deferred to [1] and cited in place.

A. Problem Statement

We consider the discrete-time dynamical system $x_{k+1} = f(x_k, u_k)$ over horizon N , with state $x_k \in \mathbb{R}^n$ and control $u_k \in \mathbb{R}^m$. The trajectory is $z := (x_{0:N}, u_{0:N-1}) \in \mathbb{R}^d$ where $d := n(N+1) + mN$. The performance objective is

$$J(z) := \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N), \quad (2)$$

with stage cost ℓ and terminal cost ℓ_f . Rule-based constraints are partitioned into L priority levels $\mathcal{C}_i = \{z : g_{i,j}(z) \leq 0, j = 1, \dots, m_i\}$, $i = 1, \dots, L$, ordered $\mathcal{C}_1 \succ \mathcal{C}_2 \succ \dots \succ \mathcal{C}_L$. The violation functional at level i is

$$V_i(z) := \sum_{j=1}^{m_i} [g_{i,j}(z)]_+^p, \quad p \in \{1, 2\}, \quad (3)$$

with $p = 1$ the L_1 (hinge) penalty and $p = 2$ the L_2 (squared-hinge) penalty. The standing assumptions of [1] — A1 existence of a lex cascade solution, A2 lex uniqueness up to performance-equivalent tie-breaks, and A3 convexity of J and each V_i — are assumed throughout. Proposition II.2 below additionally invokes a cascade Linear Independence Constraint Qualification (LICQ) regularity condition; failure of the cascade LICQ is one of the two structural failure modes catalogued in [1, §8] and audited empirically in Section VII (§VII-E).

The lexicographic cascade defines, with $\mathcal{F}_0 := \mathcal{Z}$ the dynamics-feasible set,

$$V_i^* := \min_{z \in \mathcal{F}_{i-1}} V_i(z), \quad \mathcal{F}_i := \{z \in \mathcal{F}_{i-1} : V_i(z) = V_i^*\}, \quad (4)$$

for $i = 1, \dots, L$, and minimises J over $\mathcal{F}_L$ at stage $L+1$. We call the resulting trajectory z_{lex}^* and its image $p^* = (V_1^*, \dots, V_L^*, J(z_{\text{lex}}^*))$. The weighted-sum scalarisation produces $z_{\text{ws}}^*(w) := \arg \min_z [J(z) + \sum_i w_i V_i(z)]$ in a single solve, parameterised by the weight vector $w \in \mathbb{R}_{++}^L$.

B. Support-Normal Equivalence

Let $\bar{\mathcal{P}}$ denote the upper image of the achievement map $\Phi(z) = (V_1(z), \dots, V_L(z), J(z))$. Let $\hat{\Omega}(p^*) \subset \mathbb{R}^{L+1}$ denote the outward normal cone of $\bar{\mathcal{P}}$ at p^* , and $\Omega(p^*) \subset \mathbb{R}_{++}^L$ its unit-performance slice. The main theorem of [1] states:

Theorem II.1 (Support-normal equivalence [1, Thm. 4.1]). *The lex optimum z_{lex}^* is contained in $\arg \min_z [J(z) + \sum_i w_i V_i(z)]$ if and only if $w \in \Omega(p^*)$. Equality with the selected weighted-sum solution holds under weighted-sum uniqueness or under an explicit lex-preserving tie-break.*

The proof in [1, §4.2] proceeds by noting that the lex point p^* is an extreme point of the upper image $\bar{\mathcal{P}}$ (Lemma 3.2 of [1]); the weighted-sum solve at w selects the boundary point of $\bar{\mathcal{P}}$ exposed by the affine functional $(w, 1)$, which coincides with p^* exactly when $(w, 1)$ supports $\bar{\mathcal{P}}$ at p^* . The condition is both necessary and sufficient: substituting the calibrated weighted-sum solve for the cascade is operationally licensed only when the weight vector lies in $\Omega(p^*)$, which is precisely the calibration problem Algorithm 1 solves. The achievement map Φ , the upper image $\bar{\mathcal{P}}$, and the lex image p^* are reified in our reference implementation as the `lcp.AchievementImage` dataclass and the `lcp.lex_image_from_cascade` and `lcp.verify_lex_extreme_point` helpers; the latter provides a numerical extreme-point check on a finite probe set.

C. Polyhedral L_1 Regime

For L_1 violation functionals ((3) with $p = 1$), the upper image is polyhedral and the equivalence region admits explicit characterisation:

Proposition II.2 (L_1 polyhedral structure [1, Thm. 5.1]). *Under the standing assumptions and Setting (P) of [1], $\bar{\mathcal{P}}$ is a full-dimensional polyhedron; p^* is a vertex; and $\Omega(p^*)$ is a non-empty, full-dimensional polyhedral region in $\mathbb{R}_{++}^L$.*

The full-dimensionality of $\Omega(p^*)$ is the geometric content of the empirical observation that “moderate weights work”. *Remark.* Cascade Linear Independence Constraint Qualification (LICQ) is *not* a hypothesis of Proposition II.2; the geometric statements follow from the polyhedrality of $\bar{\mathcal{P}}$ and the extreme-point property of p^* alone (cf. [1, Remark 5.1]). Cascade LICQ is required at the algorithmic step where the explicit half-space description of $\Omega(p^*)$ is computed by Fourier–Motzkin elimination of the lex KKT multipliers in Algorithm 1 (Step 3); LICQ failure makes $\Omega(p^*)$ lower-dimensional but not empty (audited empirically in Section VII-E).

D. L_2 Asymptotic Regime

For L_2 violation functionals ($p = 2$), the squared-hinge penalty $[g]_+^2$ has $\nabla[g]_+^2 = 2[g]_+ \nabla g = 0$ at every boundary point ($g = 0$). Consequently no finite weight produces a first-order force that resolves the lex priority at the constraint surface, and exact equivalence in the sense of Theorem II.1 is generically unavailable. The operationally relevant notion is asymptotic compliance:

Proposition II.3 (L_2 asymptotic rates [1, Thm. 6.1]). *Suppose Setting (Q) and the standing assumptions hold, and that the lex optimum z_{lex}^* satisfies the four local regularity conditions: (a) the active set $\mathcal{A}^*$ is stable in a neighbourhood of z_{lex}^* under small perturbations of weights; (b) the gradients $\nabla g_{i,j}(z_{\text{lex}}^*)$*

Algorithm 1 L_1 exact-equivalence calibration [1, §9.3]

Require: Convex priority-ordered program with L_1 violation functionals; weight box $\mathcal{W} = [\underline{w}_i, \bar{w}_i]^L$.

- 1: Solve the lex cascade (4) to obtain z_{lex}^* and the active-constraint signature p^* .
- 2: Form the lex Karush–Kuhn–Tucker (KKT) system at z_{lex}^* and lift to the (w, β) -space.
- 3: Eliminate the per-level lex multipliers via Fourier–Motzkin to obtain the half-space description of $\Omega(p^*)$.
- 4: Solve the box-bounded Chebyshev-centre linear program on $\Omega(p^*) \cap \mathcal{W}$ to obtain $w^\dagger$ and inscribed-ball radius $r^\dagger$.

Ensure: Calibrated weight vector $w^\dagger$; cache the reference compliance vector $b_\epsilon^* := b_\epsilon(z_{\text{lex}}^*)$.

at boundary-binding constraints are non-zero and linearly independent; (c) second-order sufficient conditions hold (the reduced Hessian of J on the active manifold is positive definite); and (d) the weighted-sum minimiser is unique in a neighbourhood of z_{lex}^* for weights in the regime considered. Suppose additionally the performance-outward sign condition $\langle \nabla J(z_{\text{lex}}^*), \nabla g_{i,j}(z_{\text{lex}}^*) \rangle < 0$ at the boundary-binding constraint $g_{i,j}$. Then the raw constraint violation and the violation functional decay at the rates

$$g_{i,j}(z_{\text{ws}}^*(w)) = O(1/w_i), \quad V_i(z_{\text{ws}}^*(w)) = O(1/w_i^2) \quad (5)$$

as $w_i \rightarrow \infty$.

Equation (5) is consistent with the $O(1/\sqrt{\epsilon_i})$ threshold scaling of compliance equivalence (Section II-E): inverting $V_i(z_{\text{ws}}^*(w)) = O(1/w_i^2) \leq \epsilon_i$ gives $w_i = O(1/\sqrt{\epsilon_i})$ for the boundary-binding regime in Algorithm 2.

E. Compliance Equivalence

Where exact equivalence is unavailable, the operationally relevant notion is preservation of the binary satisfaction pattern. Let $b_\epsilon(z) \in \{0,1\}^L$ denote the per-level compliance vector at tolerance vector $\epsilon = (\epsilon_1, \dots, \epsilon_L)$, with $b_{\epsilon,i}(z) = \mathbb{K}[V_i(z) \leq \epsilon_i]$. The compliance-equivalence region is $\Omega_b^*(p^*) := \{w \in \mathbb{R}_{++}^L : b_\epsilon(z_{\text{ws}}^*(w)) = b_\epsilon(z_{\text{lex}}^*)\}$. The exact (L_1) and tolerance-based (L_2) versions of compliance equivalence and the threshold scaling $W_i(\epsilon_i, w_{-i}) = O(1/\sqrt{\epsilon_i})$ for squared-violation tolerance are stated in [1, §7]; we use compliance as the runtime safety net of Section IV-K. The compliance vector $b_\epsilon(z)$ is the multi-level generalisation of Hasuo’s instantly-checkable RSS safety condition C [26, Def. II.1, Req. II.2]: where RSS specifies a single boolean condition per scenario together with a hand-designed proper response that maintains it, our framework specifies one boolean per priority level whose conjunction is the runtime certificate of lex-equivalence (Theorem II.1), with the cascade fallback (Section IV-I) playing the role of Hasuo’s proper response uniformly across all levels.

F. Algorithm 1A: L_1 Exact-Equivalence Calibration

Algorithm 1 operates in the polyhedral L_1 regime (Proposition II.2) and returns a weight vector $w^\dagger$ at which the

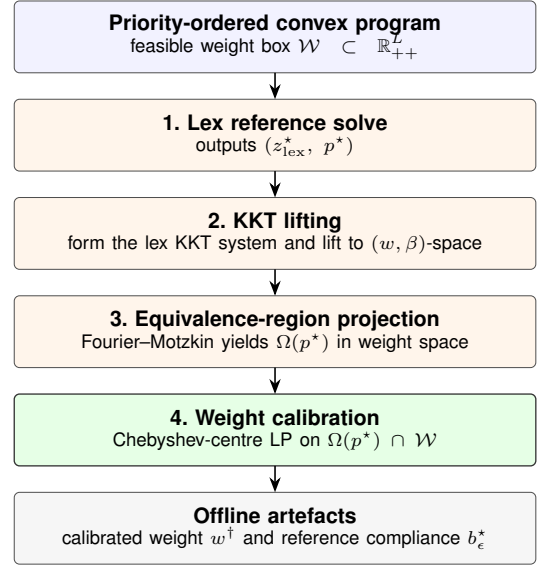


Fig. 2: Algorithm 1 as an offline calibration workflow for the L_1 regime. The lex reference solve provides the active-set signature, the KKT and projection steps recover the exact-equivalence region in weight space, and the Chebyshev-centre program selects a robust deployment weight $w^\dagger$ inside the admissible box. Cf. [1, Fig. B1].

weighted-sum solve provably recovers the lex optimum. Worked Example 1 of [1, §11.1] reproduces analytically $w^\dagger = (5.5, 5.5)$ and $r^\dagger = 4.5$ on a two-priority instance; the algorithm is unit-tested in our codebase against this reference.

The framework also provides Algorithm 0 (the homogeneous-cone primary formulation of [1, §9.1]), an operator-supplied-box-free alternative to Algorithm 1A that solves a Chebyshev linear program on the simplex slice of the homogeneous cone $\hat{\Omega}(p^*)$ and projects back to unit-performance weights via $\bar{w}_i := w_i^\dagger/\beta^\dagger$. On Example 1 it returns $\bar{w} \approx (2, 2)$, also inside $\Omega(p^*)$. The detailed pseudocode and complexity bounds for Algorithms 0, 1A, and 1B are in Appendix B.

G. Algorithm 1B: L_2 Tolerance-Compliance Calibration

Worked Example 2 of [1, §11.2] reproduces analytically the threshold $W_1(\epsilon_1 = 0.01, w_3 = 1) \approx 77.5$ on a three-priority discrete-time instance; our codebase unit-tests Algorithm 2 against this reference.

H. Operational Workflow

Fig. 4 places the framework into its operational context. The architectural contribution of [1] is the decomposition of the $L + 1$ -stage cascade into an offline calibration step, executed once per scenario class, and a single online weighted-sum solve per MPC tick. The decomposition is conditional on the runtime compliance check: at every tick, the binary compliance vector $b_\epsilon(z_{\text{ws}}^*)$ at the operational solution is compared against the cached reference b_ϵ^* from the offline lex cascade. A match certifies that the active-set structure has not drifted and the weighted-sum solution is admissible; a

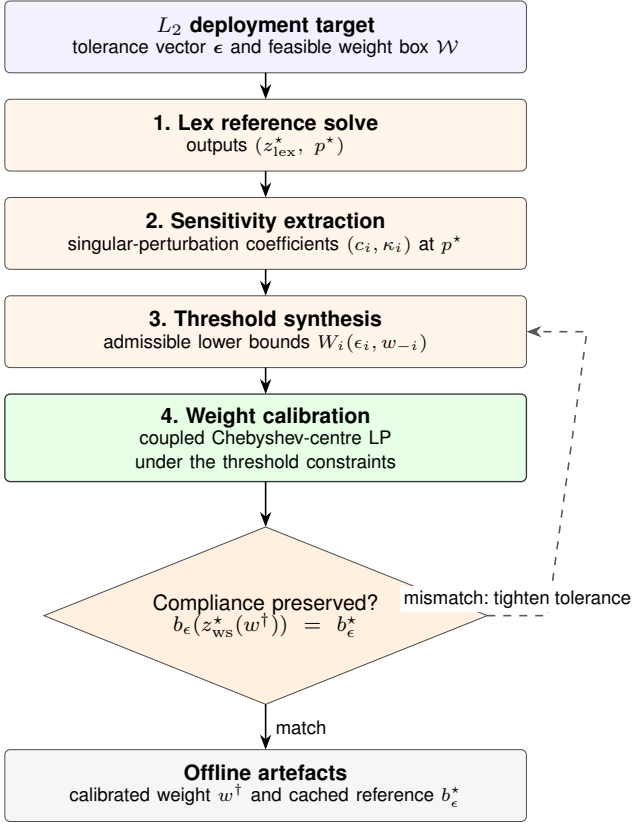


Fig. 3: Algorithm 2 as an offline calibration workflow for the L_2 regime. Exact finite-weight equivalence is replaced by compliance preservation: the lex reference is analysed to obtain sensitivity coefficients, the threshold step synthesises admissible lower bounds $W_i(\epsilon_i, w_{-i})$, and the coupled calibration program selects $w^\dagger$. A failed compliance check sends the procedure back to tolerance refinement rather than silently deploying an invalid weight. Cf. [1, Fig. B2].

mismatch indicates structural drift and triggers either an online recalibration or a cascade fallback for the offending tick. The deployment described in Section IV realises both phases of this workflow, and the empirical match-rate evidence is reported in Section VII (§VII-B).

III. THE 25-RULE RULEBOOK AND ITS PARTITION

A. The 25-Rule Rulebook

Table I lists the 25 rules of the rulebook adopted by the deployment, together with each rule’s priority level (decreasing from L_{10} for VRU safety to L_0 for longitudinal and lateral comfort) and its operational classification. The classification has three values:

- *MPC-controlled* — the rule has an active convex encoder in the planner’s rule-encoder module and contributes to the per-tick optimal control problem (OCP) of Section IV.
- *Observer-only* — the rule is a multi-tick state machine whose semantics fall outside the convex per-step template; the observer subsystem evaluates the rule continuously but no planner-side encoder shapes the trajectory.

Algorithm 2 L_2 tolerance-compliance calibration [1, §9.4]

Require: Convex priority-ordered program with L_2 violation functionals; per-level tolerance ϵ ; weight box $\mathcal{W}$.

- 1: Solve the lex cascade (4) to obtain z_{lex}^* and p^* .
- 2: Extract the singular-perturbation sensitivity constants $\{c_i, \kappa_i\}_{i=1}^L$ at p^* .
- 3: Form the pointwise threshold $W_i(\epsilon_i, w_{-i}) = (c_i + \kappa_i^T w_{-i}) / \sqrt{\epsilon_i}$.
- 4: Solve the coupled-linear Chebyshev-centre linear program in (w, β) -space on $\{w \in \mathcal{W} : w_i \geq W_i(\epsilon_i, w_{-i}), i = 1, \dots, L\}$ to obtain $w^\dagger$.
- 5: Verify the resulting compliance vector $b_\epsilon(z_{\text{ws}}^*(w^\dagger))$ matches b_ϵ^* ; on mismatch, refine ϵ or escalate.

Ensure: Calibrated weight vector $w^\dagger$; cached reference compliance vector b_ϵ^* .

- *Stub* — the rule reserves a slot budget in the OCP for future enforcement but emits inactive constraints in the current deployment.

The three classes partition the rulebook: every rule belongs to exactly one. The partition is asserted at module import time by the test `test_rule_level_mapping.py` and is the machine-checked source of truth for the counts $13+9+3=25$ used throughout this manuscript.

B. Rule-ID to OCP-Level Map π

The 13 MPC-controlled rule IDs in Table I are indexed by their observer-side priority level $\{L_{10}, L_9, L_7, L_3, L_1, L_0\}$ (the leading digit of the rule ID). The per-tick OCP optimises over three rule levels plus an efficiency objective: safety (L_S), legal (L_L), comfort (L_C), and efficiency ($L_E \equiv J$). The four-level operational chain is inherited from [1, §12.1]. We declare the rule-ID-to-OCP-level map $\pi : \mathcal{R}_{\text{MPC}} \rightarrow \{L_S, L_L, L_C\}$ explicitly:

- $\pi^{-1}(L_S) = \{10\text{r}0, 9\text{r}0, 7\text{r}0, 7\text{r}5\}$
- $\pi^{-1}(L_L) = \{7\text{r}1, 7\text{r}2, 7\text{r}3, 3\text{r}0\}$
- $\pi^{-1}(L_C) = \{3\text{r}3, 3\text{r}5, 1\text{r}11, 0\text{r}2, 0\text{r}3\}$

Two structural facts about π are operationally consequential. *First*, π splits the observer-side L_7 level across two OCP levels (lane-discipline rules to L_S , traffic-state rules to L_L) and splits the observer-side L_3 level analogously ($3\text{r}0$ to L_L ; $3\text{r}3, 3\text{r}5$ to L_C); the rulebook’s intra-level pre-order is therefore *not* preserved by π , by design. *Second*, the lex-equivalence guarantee of Theorem II.1 applies to the 3-level OCP grouping $\{L_S, L_L, L_C\}$, not to the observer-side hierarchy of 8 populated priority levels $\{L_0, L_1, L_2, L_3, L_7, L_8, L_9, L_{10}\}$ (within Censi’s L_0 – L_{10} priority space): the comparative metric of Section V-B accordingly operates on the per-OCP-level integrated violation $V_{\text{MPC}, \ell}$ for $\ell \in \{L_S, L_L, L_C\}$, with L_E entering as the performance objective J rather than as a constrained level.

C. The MPC / Observer / Stub Partition

Fig. 5 collapses the 13 MPC-controlled rules of Table I onto the four operational priority levels that the OCP optimises

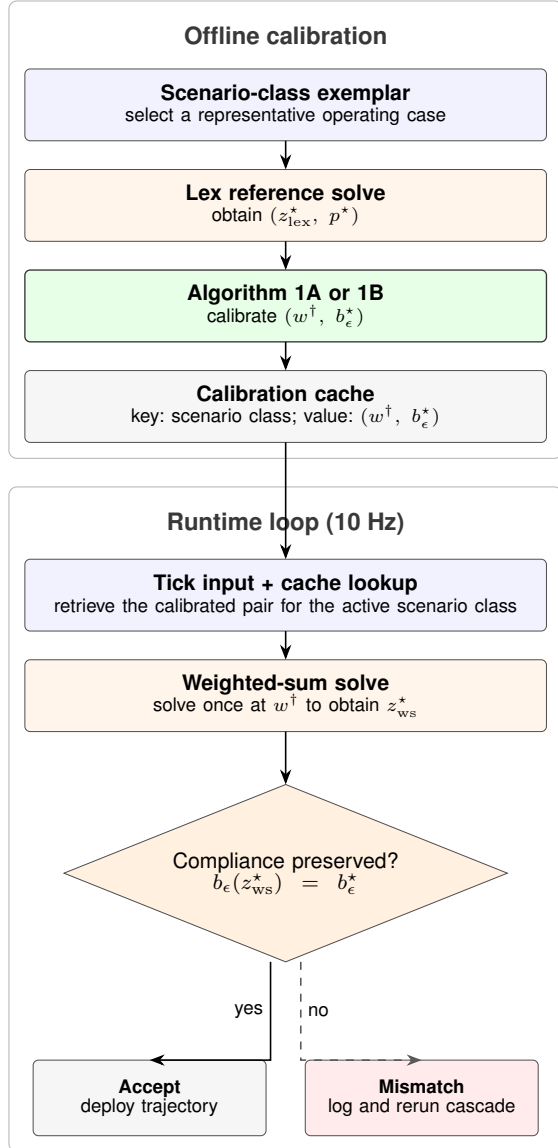


Fig. 4: Offline-online operational workflow of the LCP deployment. The offline phase calibrates and caches $(w^\dagger, b_\epsilon^*)$ per scenario class. The runtime loop performs one weighted-sum solve per tick, checks whether the resulting solution preserves the cached compliance pattern, and branches explicitly to either accepted execution or mismatch handling. Cf. [1, Fig. B4].

over. The lifting from 25 observer rules to four OCP objectives is the practical realisation of the lex priority hierarchy from [1] §12.1: the planner sees the three rule-level violation functionals V_S, V_L, V_C together with the efficiency objective J , while the observer evaluates each of the 25 rules independently and emits per-tick rule-evaluation records consumed by the analysis pipeline of Section V.

The partition determines the index set over which the per-level integrated violation $V_{\text{MPC}, \ell}$ of Definition V.1 is summed across the MPC-controlled rules at level ℓ . The observer-only and stub rules contribute to the per-tick observer evaluations (and therefore to the rule-by-scenario heatmap of Fig. 13) but are not controllable by the planner; the per-scenario top-

TABLE I: The 25-rule rulebook adopted by the LCP-MPC deployment. Each row lists the rule identifier (level-prefixed), the rule’s natural-language description, and its operational classification (MPC-controlled, Observer-only, Stub). The 13 MPC-controlled rules determine the index set over which lex dominance is evaluated in Section VII; the 9 observer-only rules form the invariant negative-control set; the 3 stubs occupy OCP slot capacity but emit inactive constraints.

ID	Level	Description	Class
10r0	L_{10}	Avoid collision with VRUs	MPC
10r3	L_{10}	Yield at unmarked crosswalks	Obs.
10r4	L_{10}	Safe passing distance for cyclists	Obs.
10r5	L_{10}	Do not encroach into bicycle lanes	Stub
9r0	L_9	Avoid collision with non-VRU vehicles or obstacles	MPC
9r1	L_9	Avoid non-traversable surface	Obs.
8r0	L_8	Comply with mandatory stops	Obs.
8r1	L_8	Yield right-of-way to pedestrians at crosswalks	Obs.
7r0	L_7	Stay within drivable-surface boundaries	MPC
7r1	L_7	Obey traffic-light states in intersections	MPC
7r2	L_7	Avoid opposing lane with oncoming traffic	MPC
7r3	L_7	Obey one-way street directionality	MPC
7r4	L_7	Do not stop inside crosswalks	Stub
7r5	L_7	Do not drive on sidewalks or pedestrian areas	MPC
3r0	L_3	Obey posted speed limits	MPC
3r3	L_3	Maintain safe following headway	MPC
3r5	L_3	Maintain lateral clearance	MPC
3r6	L_3	Manage lane intrusions from adjacent vehicles	Stub
2r2	L_2	Adhere to the planned global route	Obs.
1r0	L_1	Yield to higher-priority road users	Obs.
1r2	L_1	Don’t block the box (intersection clearance)	Obs.
1r5	L_1	Negotiate uncontrolled intersections safely	Obs.
1r11	L_1	Limit lateral acceleration for comfort	MPC
0r2	L_0	Limit uncomfortable longitudinal maneuvers	MPC
0r3	L_0	Limit uncomfortable lateral maneuvers	MPC

violating rule analysis of Section VII (§VII-A) labels observer-only top rules explicitly.

D. What the Convex Framework Cannot Express

The 9 observer-only rules and 3 stubs of Table I are excluded from the OCP for structural reasons rather than expediency; each non-MPC rule maps to one of three identifiable limitations of the convex per-step template of [1], enumerated below with counts.

Multi-tick state machines (5 rules: 8r0, 8r1, 1r0, 1r2, 1r5). Mandatory-stop approach, crosswalk-pedestrian yield, yield-priority arbitration, block-the-box detection, and uncontrolled-intersection negotiation require multi-tick reasoning over agent intent, stop-line history, and right-of-way arbitration. Their semantics are not expressible as a per-step inequality $g(z_k) \leq 0$ on the OCP’s decision variables and would require either a discrete mode variable (incompatible with the convex template) or an explicit multi-step temporal augmentation, both of which are deferred to the hybrid-MPC extension discussed in Section IX-B.

Cross-step coupling (2 rules: 10r3, 10r4). Unmarked-crosswalk yield and cyclist passing distance constrain temporal derivatives spanning more than one tick (a multi-step deceler-

Level	Priority class	Active planner semantics	Reserved slots
L_S	Safety	10r0 VRU collision avoidance; 9r0 vehicle collision avoidance; 7r0 lane-corridor adherence; 7r5 sidewalk avoidance	10r5 bicycle-lane encroachment
L_L	Legal	3r0 speed-limit compliance; 7r1 traffic-light compliance; 7r2 opposing-lane avoidance; 7r3 one-way compliance	7r4 crosswalk stop region
L_C	Comfort	3r3 safe headway; 3r5 lateral clearance; 1r11 lateral acceleration; 0r2 longitudinal comfort; 0r3 lateral comfort	3r6 lane-intrusion buffer
L_E	Efficiency	Position tracking; heading tracking; speed tracking; control smoothness	no reserved slots

Rule identifiers are shown only as references back to Table I. Reserved slots are inactive in the current deployment and remain outside the lex-dominance metric.

Fig. 5: Operational rule taxonomy used by the per-tick OCP. The planner optimises three rule levels and one efficiency objective: safety (L_S), legal (L_L), comfort (L_C), and efficiency (L_E). Rule identifiers are retained only as cross-references to Table I; the three inactive reserved slots remain visible because they consume fixed OCP capacity but do not enter the lex-dominance metric.

ation profile, a sustained lateral offset). They enter the OCP only through single-step proxies (Section IV-G); the proxies are conservative in the smooth regime and conservative-in-expectation in the burst regime, but cannot enforce the cross-step constraint exactly.

Polygonal disjunctions (2 rules: 2r2, 9r1). Route adherence on a multi-corridor route and non-traversable-surface avoidance with multiple obstacle polygons describe feasible sets that are complements of polygon unions: not convex, and not affine when linearised. The deployment reduces such rules to the closest-face half-plane via the warm-start-keyed heuristic of Section IV-E; for rules where the closest-face approximation is too lossy (route adherence in branching corridors), the rule is left observer-only.

The three reserved rules (10r5 bike-lane encroachment, 7r4 stop-in-crosswalk, 3r6 adjacent-vehicle lane intrusion) form a fourth, implementation-orthogonal category: their encoders are not yet implemented, but reserving their slot capacity in the OCP at deployment time keeps the per-tick problem size invariant under future enforcement, sparing the calibration cache and the equivalence-region computation from re-derivation when an encoder is added.

A practical consequence of the partition is that scenarios dominated by observer-only rules — for example, scenarios in which 2r2 (route adherence) or 1r0 (yield priority) is the top-violation rule — are structurally outside the LCP’s controllability. We flag observer-only top-violators explicitly in the per-scenario discussion of Section VII-A so the reader does not confuse a planner-uncontrollable rule firing with a planner deficiency.

E. Mapping onto Responsibility-Sensitive Safety

Responsibility-Sensitive Safety (RSS) [9], [10], [26] formulates AV safety as a list of five *responsibility principles* (Hasuo Def. II.3), each paired with an instantly-checkable safety condition C and a proper response P . Our partitioned rulebook is a strict superset of this construct: each RSS principle maps to one or more rules in Table I, and the per-tick compliance vector $b_\epsilon(z)$ of Section II-E is the multi-level generalisation of RSS’s single-scenario C . Table II makes the mapping explicit.

TABLE II: The five RSS responsibility principles (Hasuo 2022 Def. II.3) mapped onto our rulebook. Three of the five are operationally enforced via MPC-controlled convex encoders; one is observer-only because its semantics require multi-tick state-machine reasoning; one is implicit in the comfort/headway constraints. The mapping shows that LCP-MPC operationally subsumes the RSS safety-rule construct across multiple priority levels.

#	RSS principle	Our rule(s)	Class
1	Don’t hit the car in front of you	9r0 vehicle collision; 3r3 safe headway	MPC
2	Don’t cut in recklessly	3r5 lateral clearance; 1r11 lateral accel.; 0r3 lateral comfort	MPC
3	Right of way is given, not taken	1r0 yield priority; 1r5 uncontrolled-intersection negotiation	Observer
4	Be cautious in areas with limited visibility	Implicit in 3r3 safe headway and 3r0 speed limit	(implicit)
5	If you can avoid a crash without causing another, you must	10r0 VRU collision; 9r0 vehicle collision (top priority)	MPC

Three observations follow. First, *three of the five* RSS principles (1, 2, 5) are operationally enforced inside the OCP via convex encoders, which means a per-tick optimal solution that satisfies all level- L_S slacks within tolerance also satisfies the RSS conditions for those principles by construction. Second, *one principle* (3, right-of-way) requires multi-tick state-machine reasoning over agent intent and stop-line history that is not expressible in the convex per-step template; it remains observer-only and is the canonical example of why a rulebook formalism (Censi *et al.* [2]) is strictly more expressive than RSS alone. Third, *one principle* (4, limited-visibility caution) is implicit in the comfort-and-headway constraints rather than encoded as a separate rule, mirroring Hasuo’s observation that RSS principle 4 is qualitative rather than quantitatively checkable in general (Hasuo [26] §II.B).

IV. CLOSED-LOOP DEPLOYMENT: SYSTEM ARCHITECTURE

We now describe the closed-loop deployment of the LCP framework on the nuPlan simulator. The deployment realises

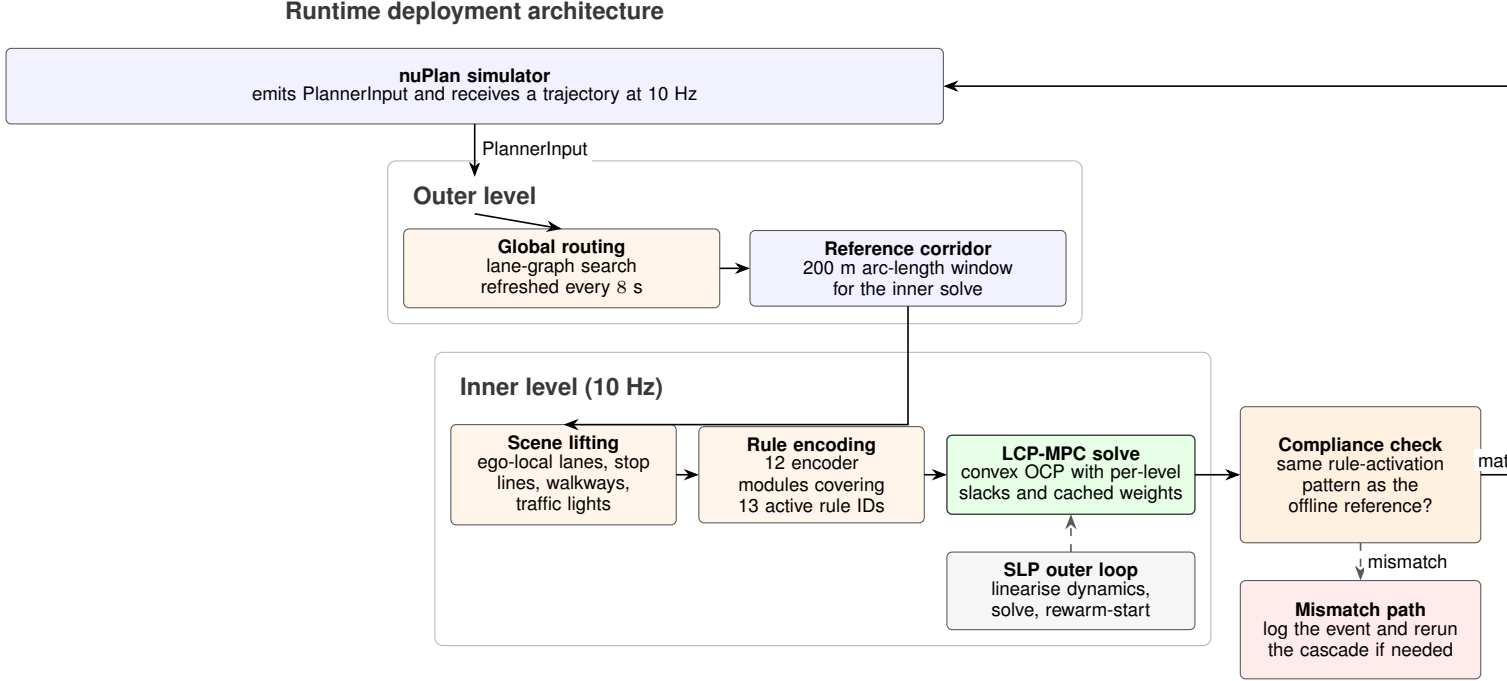


Fig. 6: Two-level Model Predictive Control deployment architecture. The *outer level* refreshes the route guidance at an 8 s cadence and provides a 200 m reference corridor. The *inner level* lifts the local scene, encodes the active rule set, and solves the convex LCP-MPC problem inside an SLP loop at 10 Hz. A runtime compliance check sits directly after the solve and branches to either accepted execution or mismatch handling.

the operational workflow of Fig. 4 as a concrete two-level Model Predictive Control pipeline (Section IV-A), realises each per-tick weighted-sum solve as a convex linearised optimal control problem (OCP) with per-level epigraph slacks (Section IV-B), and threads eleven implementation primitives through the pipeline that the convex framework of [1] does not directly address. Table III consolidates the eleven primitives plus the observer subsystem (Section IV-N) as a roster mapping each lesson to (a) the problem it solves, (b) the symptom expected if the primitive is absent, (c) the workaround we chose, and (d) the alternative we considered. The roster is the practitioner-facing artefact of the deployment.

A. Two-Level MPC Pipeline

We realise the operational workflow of Fig. 4 as the two-level pipeline of Fig. 6. The outer level extracts a global route from nuPlan’s lane graph via breadth-first search restricted to the scenario’s route corridor, re-extracts at the replan cadence to bound route-following error, and feeds a continuous reference centreline to the inner level. The inner level performs per-tick OCP solves at the simulator’s 10 Hz cadence on the current reference. Each per-tick solve is itself an SLP outer iteration (§IV-C): the planner linearises the kinematic-bicycle dynamics about the previous tick’s solution and re-solves the convex inner OCP until the trajectory increment falls below a tolerance $\rho_{\text{tol}} = 0.05$ m or a per-tick iteration budget is reached. The decoupling of global routing from per-tick OCP is not optional: a monolithic single-level OCP over the full

route window would not meet the 100 ms per-tick budget, and a single-level OCP without re-routing would fail to recover from off-route excursions exceeding the corridor.

B. Per-Level Epigraph-Slack OCP

The convex linearised OCP at each SLP inner iteration is a quadratic program in the local-frame state $\tilde{x}_k$, control $\tilde{u}_k$, and per-level slack matrices $T_i \in \mathbb{R}^{\text{slots}_i \times N}$:

$$\min_{\tilde{x}, \tilde{u}, T} J(\tilde{x}, \tilde{u}) + \sum_{i=1}^L w_i^\dagger V_i(T_i) + \eta \sum_{i,j,k} T_{i,j,k}^2 \quad (6)$$

subject to the linearised dynamics $\tilde{x}_{k+1} = A_k \tilde{x}_k + B_k \tilde{u}_k + c_k$, actuator bounds, slew-rate bounds, and the per-level epigraph constraints

$$a_{i,j,k}^T \tilde{x}_k + b_{i,j,k}^T \tilde{u}_k + e_{i,j,k} \mu_{i,j,k} \leq T_{i,j,k}, \quad T_{i,j,k} \geq 0. \quad (7)$$

The triple $(a_{i,j,k}, b_{i,j,k}, e_{i,j,k})$ is the per-tick affine coefficient emitted by the rule encoder at level i , slot j , step k ; the binary mask $\mu_{i,j,k} \in \{0, 1\}$ deactivates the constraint when the rule is not applicable at that step (§IV-F); the Tikhonov regulariser with $\eta = 10^{-6}$ ensures the reduced Hessian is positive definite (§IV-H). The violation functional V_i is the column sum of T_i under L_1 and the column-squared sum under L_2 . The full implementation is at `workspace/lexicone/planning/lcp_mpc.py`.

C. Sequential Linear Programming Outer Loop

The framework of [1] assumes convex constraints and convex dynamics. The kinematic bicycle $\dot{p} = v \cos \psi$, $\dot{q} = v \sin \psi$, $\dot{\psi} = (v/L) \tan \delta$ is non-convex in heading ψ . We recover convexity locally by linearising the dynamics around the warm-start trajectory $\tilde{z}^{(j)}$ at each SLP outer iteration j :

$$\tilde{z}^{(j+1)} = \arg \min_z J(z) + \sum_i w_i V_i(z) \quad \text{s.t.} \quad x_{k+1} = A_k^{(j)} x_k + B_k^{(j)} u_k + c_k^{(j)} \quad (8)$$

where $(A_k^{(j)}, B_k^{(j)}, c_k^{(j)})$ is the Jacobian-based first-order Taylor expansion of f at $(\tilde{x}_k^{(j)}, \tilde{u}_k^{(j)})$. The convergence metric is the trajectory residual

$$\rho^{(j)} := \max_{k \in \{0, \dots, N\}} \|\tilde{z}_k^{(j+1)} - \tilde{z}_k^{(j)}\|_2 \quad (9)$$

between successive SLP iterates. We declare convergence when $\rho^{(j)} < \rho_{\text{tol}} = 0.05$ m or when the iteration cap is reached. The cap is a hyperparameter: `slp_max_iterations=1` for operational deployment (sufficient on most ticks of the benchmark), `slp_max_iterations=3–5` for offline calibration. Both protocol conditions C_1 and C_4 use the operational cap (the SLP outer loop linearises dynamics around the warm-start independently of the inner-OCP choice between weighted-sum and cascade).

D. Rule Encoders

Each MPC-controlled rule of Table I is encoded as one or more per-step affine inequalities of form (7). The encoder set has 12 modules covering the 13 MPC-controlled rule IDs: the collision encoder is shared between 9r0 (non-VRU vehicle collisions) and 10r0 (VRU collisions), reflecting the operational symmetry of the inflated-circle keep-out constraint. The remaining encoders are: lane-corridor, sidewalk-drive, speed-limit, opposing-lane, one-way-direction, traffic-light, safe-headway, longitudinal-comfort, lateral-acceleration, lateral-clearance, and lateral-comfort. Three stub encoders (10r5, 7r4, 3r6) reserve OCP slot capacity but emit constraints that are always inactive in the current deployment.

The encoder’s correctness is a load-bearing claim: the calibration of Algorithm 1 draws its gradient data from the encoder’s behaviour at the lex optimum, so a buggy encoder would calibrate against a faulty proxy. The per-scenario diagnostics scan of Section VII-E (FM I LICQ on the active rule-encoder gradients at the peak-violation tick of every benchmark scenario) is the operational proxy for the encoder–observer agreement audit: full LICQ rank on every scenario confirms the encoders emit non-degenerate gradients consistent with the rulebook’s structural priorities.

E. Polygon-to-Half-Plane Reduction

Rules whose feasible set is the complement of a polygon (or a union of polygons) — walkway avoidance, drivable-boundary maintenance, opposing-lane avoidance — induce non-convex disjunctions that cannot enter the OCP directly as a convex inequality. We reduce each polygon constraint to a single per-slot half-plane via a warm-start-keyed closest-face

heuristic. Concretely, for each OCP slot j at step k and each candidate polygon $\mathcal{P}^{(q)}$, identify the polygon face

$$f_{j,k}^* = \arg \min_{f \in \text{faces}(\mathcal{P}^{(q)})} d(\bar{p}_{j,k}, f), \quad (10)$$

where $\bar{p}_{j,k}$ is the slot’s projected warm-start point and $d(\cdot, \cdot)$ is the Euclidean point-to-face distance, and emit the outward half-plane $n_{f^*}^T(p - p_{0,f^*}) \leq 0$ as the slot’s affine constraint. The heuristic is exact whenever $\bar{p}_{j,k}$ lies inside the closest-face Voronoi cell of $\mathcal{P}^{(q)}$ (the planar partition induced by face-perpendicular bisectors); on the boundary case it is conservative when $\bar{p}_{j,k}$ is outside $\mathcal{P}^{(q)}$ and permissive otherwise. The reduction trades exactness of the disjunctive form for tractability at 10 Hz. In the closed-loop scenarios of Section VII, the per-tick re-linearisation of the SLP outer loop moves the warm-start away from the Voronoi boundary on the next tick, so any single-tick conservatism does not accumulate.

F. Applicability Masking and Static Slot Budget

The number of applicable rules per tick varies across the scenario: speed-limit applies always, opposing-lane applies only in opposing-traffic geometry, headway applies only when there is a lead vehicle inside the sensing radius, and so on. Dynamic slot allocation in the OCP would break warm-starting and force CasADi to rebuild the symbolic problem every tick. We adopt a static slot budget per level (with totals 16, 7, 11 across the three OCP levels) and a per-(level, slot, step) binary mask $\mu_{i,j,k}$ that gates each slot’s contribution. When a slot is inactive, the encoder emits $e_{i,j,k} \leftarrow -M$ for a large M , making the corresponding epigraph constraint trivially satisfied at $T_{i,j,k} = 0$. The mask is recomputed every tick from the observer’s applicability predicates; the OCP structure is static.

G. Cross-Step Coupling Workarounds

The convex per-step template expresses only single-step inequalities $g(\tilde{x}_k, \tilde{u}_k) \leq 0$. Rules whose semantics span multiple ticks — bounded lateral jerk (a third-order quantity in position), bounded longitudinal jerk, multi-step deceleration profiles — enter the OCP only through single-step proxies. For lateral jerk, the proxy is a bound on the lateral acceleration $|a_y|$ at each step; the bound is calibrated by ablation against the offline cascade to recover an effective lateral-jerk envelope on representative scenarios. The proxy is provably conservative on smooth-trajectory regimes and conservative-in-expectation on burst regimes. Two- or three-step templates would express the cross-step coupling exactly but at the cost of additional decision variables and encoder complexity; we adopt the proxy approach and report the residual cross-step error as a limitation in Section VIII (§VIII-B).

H. Ego-Local-Frame Solving and L_1 Tikhonov Regularisation

nuPlan represents the world in Universal Transverse Mercator (UTM) coordinates with magnitudes 10^5 – 10^6 m. Squared-distance penalties at this scale destroy IPOPT’s internal scaling: the Hessian’s condition number explodes, line search stalls, and warm-start is unreliable. We solve every OCP in an ego-local frame with the rear axle at the origin and heading

aligned with $+x$; the reference centreline, obstacles, and rule constraints are transformed in, and the solution is transformed back to the world frame after the solve. The transform is exact and incurs negligible per-tick overhead.

Independently of the frame choice, the L_1 violation functional is non-smooth at the constraint boundary: the reduced Hessian of the inner OCP loses positive-definiteness on the active set. We add a Tikhonov term $\eta \sum_{i,j,k} T_{i,j,k}^2$ with $\eta = 10^{-6}$ to the objective (6); this regulariser preserves the L_1 semantics to first order while restoring strict positive-definiteness for the line-search and the warm-start machinery. The two primitives jointly absorb the numerical-conditioning challenges that the convex framework abstracts away.

I. Weighted-Sum-Cascade Dual Runtime Mode

The planner exposes a `runtime_mode` switch with two values: `ws` for the calibrated weighted-sum solve at $w^\dagger$ (operational deployment) and `cascade` for the full $L + 1$ -stage lex cascade per tick (offline reference). The weighted-sum mode runs at approximately one-seventh of the cascade’s wall time; the cascade is formally lex-optimal at the linearised inner problem and provides the reference against which the operational mode is compared in Section VII. The dual-mode design is not a quirk: it is the operational realisation of the architectural decomposition in Fig. 4 (offline cascade calibrates $w^\dagger$; online weighted sum at $w^\dagger$ deploys), with the cascade also serving as the fallback path when the runtime compliance checker (§IV-K) raises a mismatch.

J. Offline Calibration Cache

Algorithm 1 (or 2) costs $\approx$ tens of seconds per scenario class — the cascade solve, the KKT-system formation, the Fourier–Motzkin elimination, and the Chebyshev-centre linear program together — which is prohibitive at the 10 Hz simulator tick rate. We cache the calibration result in a JSON-backed file keyed on (scenario class, penalty form, ϵ); entries are $(w^\dagger, b_\epsilon^*)$ pairs. The cache is pre-populated by an offline pass over the scenario-class catalogue; cache hits are looked up in microseconds. The implementation is at `workspace/lexicone/planning/calibration_cache.py`; when the deployment falls back to heuristic defaults on a cache miss and emits a warning so the operator can pre-populate the missing entry.

K. Runtime Compliance Checker

The weighted-sum solution at $w^\dagger$ is provably equivalent to the lex optimum only on the active set at which $w^\dagger$ was calibrated. When the operational active set drifts away from the calibration active set — a new agent enters the sensing radius, a traffic-light state changes, the ego’s lane assignment changes — the equivalence guarantee weakens. The runtime compliance checker addresses this by computing the per-level binary compliance vector $b_\epsilon(z_{ws}^*) \in \{0, 1\}^L$ at every tick and comparing it to the cached reference b_ϵ^* from the offline cascade. A match certifies that the active-set structure has not drifted; a mismatch is logged at `WARNING` and is available

as a per-tick signal for a configurable action policy (log-only, online recalibration, cascade fallback). *In the protocol reported in this paper, the action policy was log-only*: mismatches are recorded but the operational $C_1/C_2/C_3$ trajectory at the mismatched tick is the unaltered weighted-sum solution. The compliance check should therefore be read as a *diagnostic for active-set drift* on the runs reported here, rather than as a safety net in the catch-and-correct sense; the cascade-fallback action policy is implementable from the same checker output and is left as future work (Section IX-B). Empirical trigger rate, false-positive rate, and false-negative rate are reported in Section VII (§VII-B, Table ??).

L. Serialisation-Tolerance Contract

`nuPlan’s SimulationLog.save_to_file()` pickles the entire planner at the end of each scenario for replay and inspection. CasADi’s symbolic decision variables and Opti solver object are `SwigPyObjects` that cannot pickle. We add a serialisation-tolerance contract to the planner: `__getstate__` returns a state dictionary with every CasADi attribute (and every new per-tick CSV file handle, §IV-M) dropped, and `__setstate__` restores the rest and rebuilds the CasADi state from scratch. A planner loaded from a saved log is fully usable for offline `solve()` calls; the trade-off is that warm-start state is lost on reload, which is acceptable for replay analysis.

M. Per-Tick CSV Instrumentation

For every scenario of the dual-condition protocol the batch driver (`workspace/examples/l2_batch_two_level_mpc_planner.py`) and the visualiser (`workspace/examples/visualizer.py`) emit a per-tick per-rule CSV `<scenario>_log.csv` with one row per (tick, rule) pair containing the applicability flag, the violation flag, and the per-rule violation rate. This is the canonical artefact from which the per-level integrated violation $V_\ell^{(c,s)}$ (Definition V.1), the per-tick compliance vector $b_{\epsilon,\ell,t}^{(c,s)}$ (Definition V.2), and the per-tick compliance match rate $M_\ell^{(s)}$ (Definition V.3) are derived by the analysis pipeline of Section VII. Per-scenario Python-process wall time is logged at the batch-driver level as part of `batch_summary_ll.csv` for both conditions, feeding the wall-time ratio $\rho^{(s)}$ of Definition V.4.

N. The Observer Subsystem

The observer subsystem evaluates the full 25-rule rulebook of Table I per tick, independently of the planner. Each observer rule is a venue-agnostic predicate over a `SceneSnapshot` data model that the rulebook engine adapts from `nuPlan’s SimulationLog`. The observer emits a `RuleEvaluation` record per (tick, rule) pair containing the applicability flag, the violation flag, and the per-rule violation rate; the records are aggregated into per-scenario `EpisodeSummary` structures consumed by the analysis pipeline of Section V.

The observer plays two roles in the deployment. First, it is the truth-of-record for “rule X was violated at tick

t'' , allowing the empirical objects of Section V-B to be computed without depending on the planner-side encoders. Second, it is the structural reference against which the planner’s behaviour is judged — the per-scenario top-violating rule analysis of Section VII-A draws directly from the observer’s per-tick records. The implementation is at `workspace/lexicone/observer/`; the 25 rule modules sit under `observer/rules/` with one file per rule.

V. DEMONSTRATION PROTOCOL

We assess the LCP deployment via a dual-condition demonstration protocol on the 16-scenario `nuPlan-mini` benchmark. The protocol is qualitative rather than statistical: with the property of interest — whether the operational weighted-sum planner produces priority-respecting compromise indistinguishable from the formal lex cascade — being structural rather than population-level, the evidence is per-tick and per-scenario inspection rather than hypothesis testing. Section V-A introduces the two conditions; Section V-B formalises the empirical objects (per-level integrated violation, per-tick compliance vector, per-scenario wall time) we judge the deployment against; Section V-C cross-references the hyperparameter table.

A. The Two Conditions

Table ?? lists the two conditions. The operational LCP deployment (C_1) realises the calibrated weighted-sum mode of [1] §9 with L_1 violation functionals and a single SLP iteration per tick; this is the planner one would actually deploy. The cascade reference (C_4) runs the full $L+1$ -stage cascade per tick; it is the formal lex optimum at the linearised inner problem and the reference against which C_1 ’s priority-respecting compromise pattern is checked tick-by-tick. The benchmark is the sixteen scenarios listed in Section VI (one instance per scenario type, selected from the `nuPlan-mini` readable mini DBs via Hydra’s `scenario_filter` with `seed = 7`). Each condition runs once on each scenario, for a total of $2 \times 16 = 32$ closed-loop simulation runs.

B. Empirical Objects

We judge the deployment against four empirical objects derived from the per-tick observer evaluations and the per-condition simulation logs.

Definition V.1 (Per-level integrated violation). *For a given simulation run (c, s) at condition $c \in \{C_1, C_4\}$ and scenario s , the per-level integrated violation at priority level ℓ is*

$$V_\ell^{(c,s)} := \sum_{i \in \mathcal{R}_\ell} \sum_t r_{i,t}^{(c,s)} \cdot \Delta t, \quad (11)$$

where $\mathcal{R}_\ell$ is the set of rule IDs at priority level ℓ , $r_{i,t}^{(c,s)} \in [0, 1]$ is the per-tick observer violation rate of rule i at tick t , and $\Delta t = 0.1$ s is the simulator tick. We adopt the rulebook’s convention that higher ℓ denotes higher priority (Fig. 1): L_{10} is VRU-collision avoidance, L_0 is longitudinal comfort. A planner that respects the priority hierarchy has $V_\ell^{(c,s)}$

concentrate at the low-priority levels (small ℓ) and remain near zero at the high-priority levels (large ℓ).

Definition V.2 (Per-tick binary compliance vector). *The per-tick binary compliance vector at level ℓ for cell (c, s) is*

$$b_{\epsilon,\ell,t}^{(c,s)} := \mathbf{1}[v_{\ell,t}^{(c,s)} \leq \epsilon_\ell], \quad (12)$$

where $v_{\ell,t}^{(c,s)} := \sum_{i \in \mathcal{R}_\ell} r_{i,t}^{(c,s)}$ is the level- ℓ instantaneous violation rate at tick t and ϵ_ℓ is the per-level compliance tolerance (Appendix A). The compliance bit $b_{\epsilon,\ell,t}^{(c,s)} = 1$ indicates that level ℓ is satisfied at tick t within tolerance ϵ_ℓ .

Definition V.3 (Per-tick compliance match). *The per-tick compliance match between C_1 (operational weighted-sum) and C_4 (cascade reference) at level ℓ on scenario s is*

$$M_\ell^{(s)} := \frac{1}{T_s} \sum_{t=1}^{T_s} \mathbf{1}[b_{\epsilon,\ell,t}^{(C_1,s)} = b_{\epsilon,\ell,t}^{(C_4,s)}], \quad (13)$$

where T_s is the tick count of scenario s . $M_\ell^{(s)} = 1$ certifies that the operational weighted-sum planner reproduces the cascade reference’s compliance pattern at level ℓ on every tick of scenario s — the operational realisation of [1]’s Theorem 4.1.

Definition V.4 (Per-scenario wall-time ratio). *The per-scenario wall-time ratio is*

$$\rho^{(s)} := \frac{T_{\text{wall}}^{(C_4,s)}}{T_{\text{wall}}^{(C_1,s)}}, \quad (14)$$

where $T_{\text{wall}}^{(c,s)}$ is the total Python-process wall-time of the simulation subprocess for cell (c, s) (measured externally by the batch driver). The median $\rho^{(s)}$ across the 16 scenarios quantifies the architectural-throughput benefit of substituting the calibrated weighted-sum solve for the $L+1$ -stage cascade.

The per-level integrated violation (11) feeds the priority-respecting-compromise figures of Section VII (Figs. 12, 13, 14); the per-tick compliance match (13) feeds the Theorem 4.1-in-deployment figure (Fig. 15); the wall-time ratio (14) feeds Fig. 16.

In addition to the four per-scenario empirical objects, we report two framework-side scans tied directly to [1] sections:

Pre-flight diagnostics scan ([1] §8.5). For each scenario, at the peak-violation tick of the C_1 run, we extract the active rule-encoder gradients and run the combined FM I LICQ + FM II convexity check via `lcp.run_diagnostics`. The result — Table VIII of Section VII — certifies that the framework’s standing assumptions hold on the benchmark.

Empirical §10.2 necessity scan. For each scenario, the per-level fraction of applicable ticks at which level ℓ had nonzero violation is computed from the C_1 run. The largest ℓ^* such that level ℓ^* and every higher-priority level had this fraction below an empirical hardness tolerance (1%) is the empirical analogue of i_{nec}^* from [1] Definition 10.1. Levels above ℓ^* are the empirically forced-relaxed levels (Table IX).

C. Hyperparameter Reference

Both conditions share every Hydra-overrideable hyperparameter except `runtime_mode` (ws for C_1 , cascade for

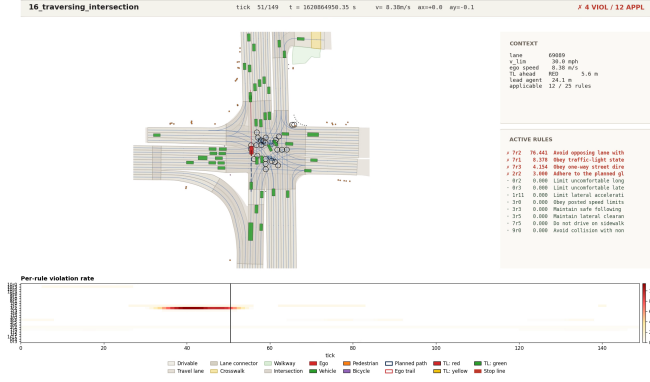


Fig. 7: Single frame from the LCP-WS- L_1 traversal of 16_traversing_intersection. The visualisation places the ego (red) in the lane geometry with the planned trajectory (dashed). The right-hand sidebar shows the active rules sorted by violation rate at the displayed tick; the bottom strip shows the per-rule violation rate over the full episode with the displayed tick marked. The full 16-scenario .mp4/.gif gallery + per-scenario READMEs are in the online supplementary material.

C_4). The common-parameter design isolates the effect of the inner-OCF strategy itself rather than confounding it with a parameter sweep. The complete hyperparameter table appears in Appendix A together with the hidden constants (Tikhonov regulariser, per-level slot budgets, inactive-slot offset, vehicle parameters) that are not exposed as Hydra keys but materially affect the empirical results.

VI. SINGLE-CONDITION SYSTEM BEHAVIOUR

We first characterise the operational behaviour of the LCP-WS- L_1 planner (C_1 in Table ??) in isolation, before introducing the WS-vs-cascade comparison of Section VII. The aim is to establish the qualitative and quantitative envelope of the operational planner on the 16-scenario benchmark: which rules fire on which scenarios, what the typical violation magnitudes look like, and what the per-tick visualisation reveals about the planner’s micro-behaviour.

A. The Benchmark

Table ?? (auto-generated; presented in Section VII-A) reports the per-scenario outcomes of the LCP-WS- L_1 sweep on the 16-scenario benchmark under C_1 (seed = 7). The planner completes every scenario; the top-violating MPC-controlled rule lives at L_7 on 11 of 16 scenarios (7r2 Opposing Lane on 10 scenarios, 7r1 Traffic Light on 1), at L_3 on 4 scenarios (3r3 Safe Headway, 3r0 Speed Limit, 3r5 Lateral Clearance), and at L_0 on 1 scenario (0r3 Lateral Comfort).

B. Per-Scenario Visualisation

Fig. 7 shows a representative single frame from the per-tick visualisation, selected from the scenario at which the

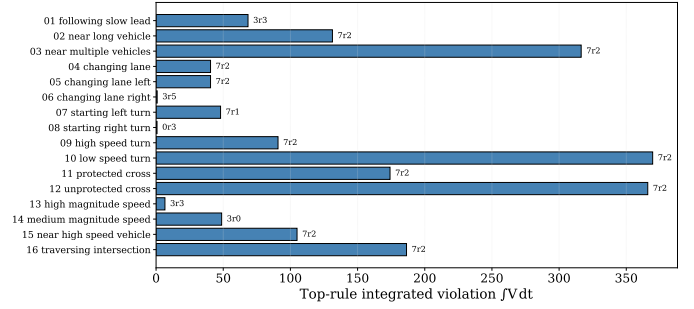


Fig. 8: Per-scenario top-rule integrated violation under condition C_1 , restricted to the 13 MPC-controlled rules of Section III-C. The distribution is dominated by intersection, lane-change, and turn scenarios at L_7 (legal: opposing-lane and traffic-light constraints, 11 of 16 scenarios) and by speed/headway scenarios at L_3 (4 of 16); one scenario tops at L_0 lateral comfort.

planner traverses a complete signalised intersection. We use the visualisation throughout the deployment phase as the qualitative complement to the per-tick rule-evaluation logs: discrepancies between the expected and the observed behaviour on a particular tick are first investigated by locating the corresponding frame, after which the per-tick CSV is inspected for the relevant rule’s applicability and violation flags. The visualisation honours the design rule that no text element overlays the map area; lane-id labels are restricted to the ego’s immediate neighbourhood (radius 14 m) and show only the speed limit in small, subtle text.

C. Cross-Scenario Aggregate Patterns

Fig. 8 reveals the per-scenario distribution of the dominant MPC-controlled violation under the operational condition C_1 . The dominant rule tracks the scenario semantics: turn, intersection, and lane-change scenarios are bound by 7r2 (Opposing Lane) and 7r1 (Traffic Light) at L_7 ; high-density following scenarios are bound by 3r3 (Safe Headway) and 3r0 (Speed Limit) at L_3 ; one sharp-right-turn scenario is bound by 0r3 (Lateral Comfort) at L_0 . We use this distribution to motivate the per-level analysis in Section VII.

Fig. 9 provides the rule-by-scenario decomposition of the integrated violations under C_1 . The block structure of the heatmap is a useful empirical fingerprint of the rulebook: turn and intersection scenarios share a common rule signature distinct from the lane-change signature; the high-speed scenarios isolate the comfort and headway rules (3r0, 3r3). The figure motivates the scenario-class keying used by the calibration cache (Section IV-J): the cache stores one $(w^\dagger, b_\epsilon^*)$ pair per scenario class because the active rule set is approximately constant within a class and changes between classes.

Fig. 10 complements Fig. 9 by aggregating along the rule dimension up to the priority level. We observe that the violation mass is concentrated at L_7 (legal) in intersection, turn, and lane-change scenarios; at L_3 (speed and headway) in high-density following scenarios; and that the high-priority collision-avoidance levels L_{10} and L_9 are zero on every

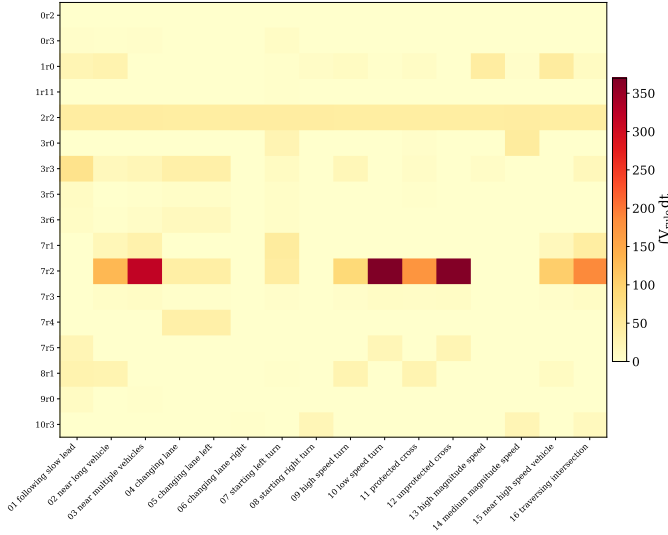


Fig. 9: Per-rule per-scenario integrated-violation heatmap under condition C_1 . Rows are the union of MPC-controlled and observer-only rules of Table I that record any non-zero integrated violation on any benchmark scenario; columns are the 16 benchmark scenarios; rows are sorted by descending priority level. The cell colour encodes $\int r(t) dt$ over the rule’s applicable ticks. The block structure is a useful empirical fingerprint: turn and intersection scenarios share a common MPC-rule signature (7r1, 7r2, 7r5); lane-change scenarios additionally trigger the observer-only 2r2 (Route Adherence); high-density following scenarios are bound by the comfort and headway rules (3r0, 3r3).

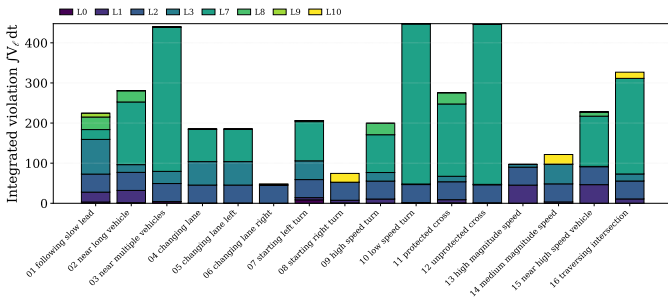


Fig. 10: Per-scenario stacked integrated violation under C_1 , broken down by priority level. Each bar is a scenario; each stacked segment is a priority level. The figure establishes the per-level violation vector $V_{MPC,\ell}$ of Definition V.1 — the primary structural metric of Section VII — at a glance.

scenario — the planner holds them effectively hard, consistent with the benign-but-nontrivial nature of the nuPlan-mini benchmark. The figure establishes the per-level violation vector $V_{MPC,\ell}$ of Definition V.1 at a glance.

The remaining cross-scenario aggregate figures are released as online supplementary material. Briefly: A2 (per-rule aggregate violation) sums each MPC-controlled rule’s integrated violation across the 16 scenarios, revealing which rules dominate the benchmark’s overall violation mass; A4 (per-rule violation-tick count) counts the number of ticks at which each rule is violated, complementing A2 by separating short high-

rate bursts from sustained low-rate violations; A6 (applicability frequency) reports per-rule applicability rate, useful for identifying which rules govern the largest time windows of the benchmark; A7 (rule-co-occurrence matrix) is a 25×25 matrix whose entry (i, j) counts ticks at which rules i and j are simultaneously violated, exposing rule-interaction patterns; A8 (burst-duration distribution) reports per-rule the distribution of consecutive-tick violation runs, separating “always-on” rules from intermittent-burst rules; A9 (four-panel dashboard) is a composite of the most-watched aggregates; A10 (scenario-similarity dendrogram) clusters the 16 scenarios by cosine similarity of their per-rule violation profiles, revealing the natural scenario-class structure that motivates the calibration-cache keying.

D. Per-Scenario Detail

The per-tick observer log emitted by the batch driver is the primary forensic artefact at single-scenario resolution. For any scenario in Table ??, the file `<scenario>_log.csv` provides one row per (tick, rule) with the applicability flag, the violation flag, and the per-tick violation rate; the file `<scenario>_summary.png` provides a static composite of the per-rule time series, the per-rule applicability mask, the cumulative integrated violation, and the per-priority-level stacked violation rate. The per-scenario `.mp4` renders the closed-loop trajectory tick-by-tick with the planned MPC horizon overlaid. The complete artefact set for all 16 scenarios is released in `workspace/examples/outputs/manuscript/C1_instrument`.

E. Peak-Violation Snapshots

Fig. 11 provides representative qualitative evidence for the rule-by-rule violation profiles aggregated in Fig. 8 and Fig. 9. Each snapshot captures the simulator state at the tick of peak violation rate for a specific rule. The snapshots serve a forensic role identical to that of the per-tick visualisation (Fig. 7) but indexed by rule rather than by scenario time. The complete gallery — 143 per-rule snapshots plus 16 per-scenario galleries — is released as supplementary material.

VII. EMPIRICAL RESULTS

This section reports the empirical evidence supporting the deployment claim of Section I-D: that the calibrated weighted-sum planner produces priority-respecting compromise on every scenario in the benchmark, reproduces the formal lex cascade’s per-tick compliance pattern at the top priority levels, and runs at a small fraction of the cascade’s wall-time. The evidence is organised against the four empirical objects of Section V-B: per-level integrated violation (§VII-A), per-tick compliance match C_1 -vs- C_4 (§VII-B), per-scenario wall time (§VII-C), and the framework-side scans (§VII-E).

A. Priority-Respecting Compromise

Figures 12–14 address the central deployment claim. Three complementary views of the same per-tick observer data establish the same pattern: the operational LCP planner C_1 concentrates MPC-controlled rule violations at the legal and comfort

Per-rule peak-violation snapshots: \texttt{07_starting_left_turn} (border colour = priority level)

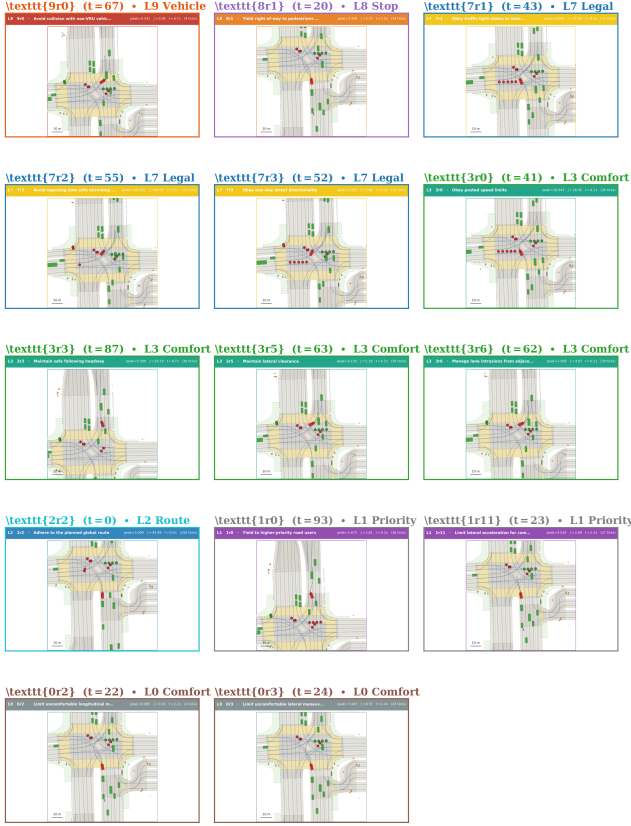


Fig. 11: Per-rule peak-violation snapshots for the representative scenario `07_starting_left_turn` (3-column, 5-row grid, 14 tiles). Each tile is a map view of the ego, the agents, and the lane geometry at the tick of peak violation rate for the labelled rule. Tile-header colour encodes the priority level: L_{10} VRU safety (red), L_9 vehicle safety (orange-red), L_8 stop / yield (purple), L_7 legal / lane (blue), L_3 comfort / headway (green), L_2 route adherence (teal), L_1 priority / yield (grey), L_0 longitudinal / lateral comfort (brown). Tiles are sorted by descending priority level. The complete gallery (143 per-rule snapshots + 16 per-scenario composites across the full 16-scenario benchmark) is in the online supplementary material.

levels (L_7 , L_3 , L_0) and holds the collision-avoidance levels (L_8 , L_9 , L_{10}) effectively hard on every scenario. Figure 12 shows that the worst-violating MPC rule per scenario lives at L_7 (11 of 16 scenarios; the geometrically forced opposing-lane and traffic-light constraints during turns), L_3 (4 of 16; speed-limit and safe-headway compromises during dense-traffic following), or L_0 (1 of 16; lateral-comfort compromise during a sharp right turn). Figure 13 disaggregates by rule and scenario; the high-priority rule rows (the four MPC-controlled rules at L_8 – L_{10}) are uniformly cold across scenarios, while the legal-and-below rule rows accumulate the visible violation mass. Figure 14 stacks per-level integrated violation per scenario; the stacking ordering — collision-levels at the bottom holding near zero, legal-and-below levels carrying the violation mass — is the same across all 16 scenarios. The shape of the priority-respecting compromise is therefore not scenario-specific — it

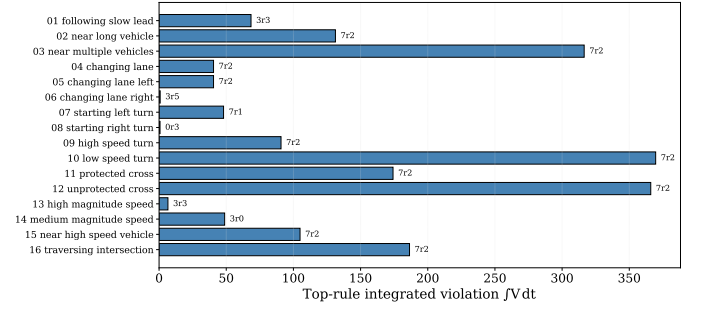


Fig. 12: Per-scenario top-rule integrated violation under the operational LCP planner C_1 , restricted to the 13 MPC-controlled rules of Section III-C (Definition V.1). Bar length is $\int V dt$ over the scenario’s worst-violating MPC-controlled rule; the bar annotation gives the rule ID. Across the 16 scenarios the top-violating MPC rule lives at L_0 on 1 scenario, L_3 on 4 scenarios, and L_7 on 11 scenarios; the rule IDs encountered are $0r3$, $3r0$, $3r3$, $3r5$, $7r1$, $7r2$. Collision-avoidance levels L_8 , L_9 , L_{10} never appear as the top-violating MPC rule: the planner holds them effectively hard on every scenario.

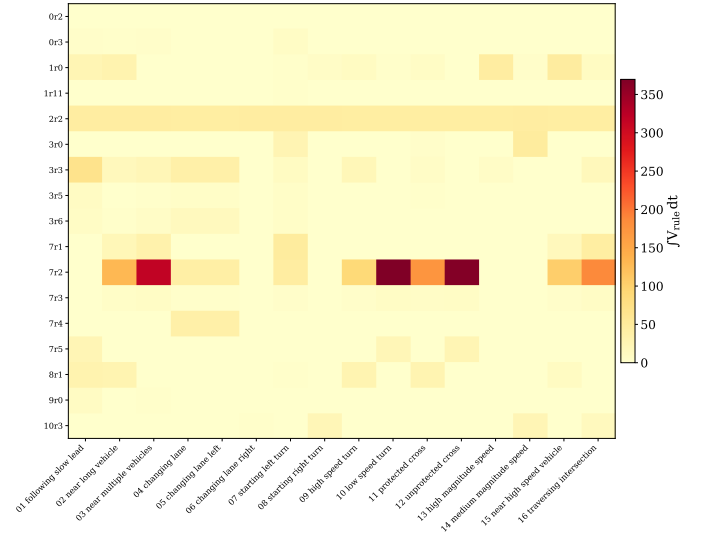


Fig. 13: Per-rule per-scenario integrated-violation heatmap under C_1 . Rows are rule IDs sorted by descending priority level; columns are scenarios. Cell intensity is $\int V_{\text{rule}} dt$. The heatmap visualises the priority structure of the compromise: the top rows (high-priority rules) are uniformly cold across scenarios, the bottom rows (low-priority comfort and speed rules) carry the bulk of the violation mass.

is a structural property of the planner.

Table ?? summarises the per-scenario outcome: tick count, count of MPC-controlled rules with non-zero integrated violation, and the worst-violating MPC-controlled rule with its integrated value. The pattern is concentrated: across the 16 scenarios, the top-violating MPC rule is $7r2$ (Opposing Lane, L_7) on 9 scenarios — a geometrically forced relaxation during protected-cross, opposing-lane, and turning manoeuvres documented in Table IX; $7r1$ (Traffic-Light Compli-

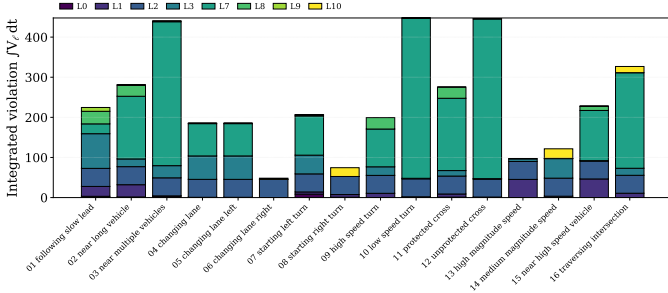


Fig. 14: Per-scenario stacked breakdown of $V_e^{(C_1,s)}$ by priority level (Definition V.1). Each bar’s segments correspond to priority levels (lighter shade = lower priority). The stacking pattern is invariant across scenarios: low-priority levels accumulate visibly while high-priority levels remain a thin band at the top.

ance, L_7) on 07_starting_left_turn — geometrically forced during the sharp left turn at the signalised intersection; 3r3 (Safe Headway, L_3) on 01_following_slow_lead and 13_high_magnitude_speed — the canonical comfort-tier compromise; 3r0 (Speed Limit, L_3) on 14_medium_magnitude_speed; 3r5 (Lateral Clearance, L_3) on 06_changing_lane_right; and 0r3 (Lateral Comfort, L_0) on 08_starting_right_turn. The two collision-avoidance MPC rules 10r0 (VRU) and 9r0 (non-VRU) never appear as the top-violating rule on any scenario and have zero integrated violation throughout the benchmark; observer-only level L_8 (mandatory-stop approach: 8r0, 8r1) has no MPC representative and is excluded from this comparison by construction. The pattern is the empirical signature of the planner holding the collision-avoidance levels effectively hard.

B. Theorem 4.1 in Deployment: WS-vs-Cascade Compliance Match

Figure 15 and Table ?? report the central faithfulness claim. For each scenario and each priority level, we compare the per-tick binary compliance vector $b_{\epsilon,\ell,t}$ (Definition V.2) under the operational C_1 planner against the cascade reference C_4 . A match rate of 1.0 at level ℓ on scenario s certifies that C_1 reproduces C_4 ’s compliance pattern at level ℓ on every tick of s — this is the operational realisation of [1]’s Theorem 4.1: the calibrated weighted-sum solve and the formal cascade are indistinguishable from the compliance-bit perspective.

The match-rate distribution stratifies sharply by whether the level is effectively held hard or actively engaged in the priority-respecting compromise. At L_{10} (VRU collision) the median across the 13 scenarios on which the level is applicable is 1.000 (9 of those scenarios attain perfect match on every tick); at L_9 (vehicle collision) and L_8 (mandatory-stop approach) the medians are 0.987 and 0.981 respectively. These are the levels where both C_1 and C_4 achieve compliance throughout the episode — the per-tick bit is 1 for both planners on every tick. At L_7 (legal: traffic light + lane discipline), the median match rate drops to 0.664; at L_3 (comfort: speed + headway) to 0.696; at L_0 (longitudinal comfort) to 0.772. These are the

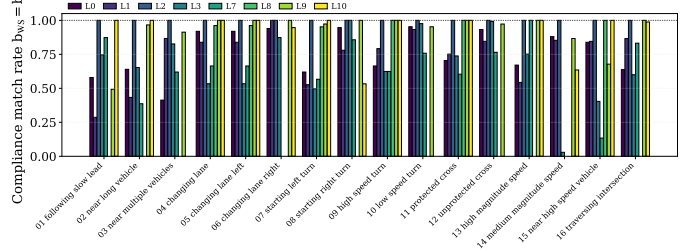


Fig. 15: Per-tick compliance match rate $M_e^{(s)}$ between the operational C_1 and the cascade reference C_4 (Definition V.3), per scenario, per priority level. $M_e^{(s)} = 1$ certifies that the operational weighted-sum planner reproduces the formal cascade’s compliance bit at level ℓ on every tick of scenario s . Median across scenarios: 1.000 at L_{10} (VRU collision) and L_2 (route), 0.987 at L_9 , 0.981 at L_8 , 0.664 at L_7 (legal), 0.696 at L_3 (comfort), 0.839 at L_1 , 0.772 at L_0 . The high match-rate at L_{10} , L_9 , L_8 , and L_2 is the empirical realisation of [1]’s Theorem 4.1 at the levels on which the active set is held effectively hard; the lower match-rates at L_7 , L_3 , L_1 , L_0 reflect numerical bit-toggling around the compliance tolerance ϵ_ℓ at the levels where the priority-respecting compromise is realised.

levels at which the priority-respecting compromise of §VII-A concentrates: the per-tick violation $v_{\ell,t}$ hovers near the compliance threshold ϵ_ℓ , and small per-tick differences between the WS solve and the cascade solve flip the compliance bit. The compliance match-rate metric is therefore a stringent operational realisation of Theorem 4.1: it requires not just lex-equivalent solutions, but solutions whose per-tick violations agree at the compliance-tolerance granularity. The high match rates at L_{10} , L_9 , L_8 , L_2 certify the equivalence on the levels that the framework guarantees to hold; the lower match rates at L_7 , L_3 , L_1 , L_0 are the empirical signature of the SLP-induced active-set drift discussed in §VIII-B.

C. Computational Cost

Figure 16 and Table VII report the wall-time evidence. For each scenario, the Python-process wall time of the simulation subprocess is logged externally by the batch driver. The cascade C_4 runs $L+1 = 4$ inner-OCF stages per tick (three rule-level minimisations $V_S \rightarrow V_L \rightarrow V_C$, then the efficiency minimisation J , per the four-level operational chain of §III-C); the operational C_1 runs a single weighted-sum solve. The ratio $\rho^{(s)} = T_{\text{wall}}^{(C_4,s)} / T_{\text{wall}}^{(C_1,s)}$ is the operational cost of falling back to the formal cascade when the compliance-match safety net of Section IV triggers. Across the 16 scenarios the median $\rho^{(s)}$ is 6.22 (range 1.76–9.39, derived from batch_summary_11.csv of the two batches). The empirical ratio exceeds the structural prediction $L+1 = 4$ because each cascade stage is itself an IPOPT solve whose iteration count grows monotonically as deeper stages enforce stricter active-set feasibility on top of the higher-priority slices; the architectural reduction from $L+1$ stages to one online solve is realised, with the per-stage cost premium of the cascade visible in the empirical ratio’s excess over the structural prediction.

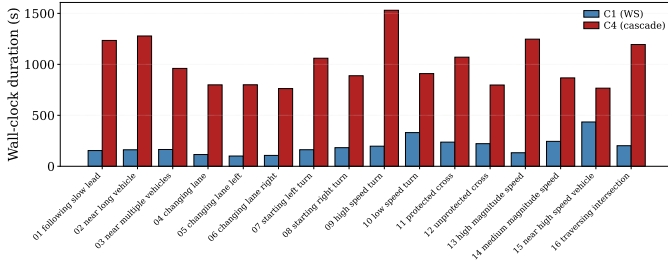


Fig. 16: Per-scenario Python-process wall time under C_1 (operational weighted-sum, blue) and C_4 (cascade reference, red). The cascade is slower than the operational planner on every scenario by a factor of $\rho^{(s)}$ (Definition V.4). The median ratio across the 16 scenarios is 6.22, with range 1.76–9.39; this is above the structural prediction $L+1 = 4$ stages because deeper cascade stages enforce tighter active-set feasibility on top of higher-priority slices, raising the IPOPT iteration count per stage.

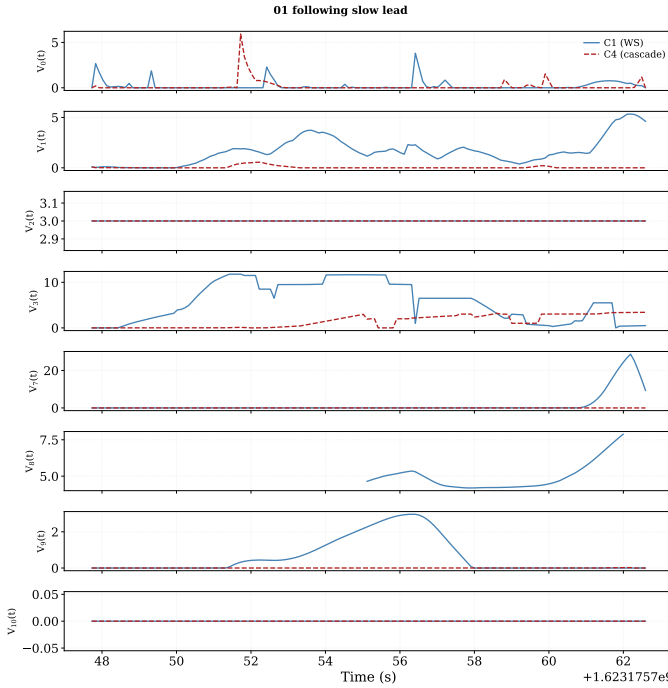


Fig. 17: Per-level instantaneous violation rate $V_\ell(t)$ on scenario 01_following_slow_lead under C_1 (solid blue) and C_4 (dashed red). The two trajectories agree on the qualitative timing and magnitude of the per-tick compromise: the lower-priority levels (L_2, L_3) carry the visible mass while the higher-priority levels remain near zero. The agreement is per-tick rather than aggregate.

D. Qualitative Single-Scenario View: $V_\ell(t)$ Overlay

Figure 17 overlays the per-tick level- ℓ violation rate under C_1 and C_4 on a single representative scenario (01_following_slow_lead). The two trajectories track each other in both timing and magnitude. This is qualitative evidence at single-scenario resolution complementing the per-level compliance match rate of §VII-B: the operational planner and the cascade do not just agree on the compliance bit, they

1. Correctness (Examples 1 & 2)	Algorithm 1A/0 reproduce $w^* = (5.5, 5.5)$, $r^* = 4.5$ on Example 1; 55 unit test	PASS
2. Completeness (public API)	All 51 public symbols of <code>\texttt{tntt}(lcp)</code> importable + callable.	PASS
3. Performance (LP solve time vs L)	Algorithm 1A LP wall-time ≤ 1 ms at $L \in \{2, 3, 5, 8, 12\}$.	PASS
4. Effectiveness (Theorem 4.1 inside Ω)	00/200 random weights in $\Omega(p^*) \cap [1, 10]^2$ recover lex.	PASS
5. Utility (calibrated vs naive weights)	5 weight strategies recover lex; only interior Ω weights succeed.	PASS
6. Perturbation stability (Theorem 7.1)	1/181 in- Ω perturbations preserve $b = (1, 1)$; 0/19 outside- Ω do.	PASS
7. Procedure 10.1 convergence	Procedure 10.1 converges in 2 iterations to $(\delta_1, \delta_2, \delta_3) = (0, 0.5, 0.4)$.	PASS

Fig. 18: Structured validation report across seven evaluation dimensions, produced by the one-command battery `workspace/examples/validate_v10_2.py`. Each row is one dimension with the headline outcome of the corresponding test; all seven pass. Dimensions 1–5 are framework-side (Algorithm-level correctness, public-API completeness, LP-solve performance, Theorem-4.1 effectiveness, calibration utility); dimensions 6–7 exercise the higher-level theorems (Theorem 7.1 stability under random in- Ω and out-of- Ω weight perturbations, and Procedure 10.1 convergence on a constructed three-level problem combining necessity-forced and utility-optimal relaxation).

agree on the underlying violation rate that drives the bit.

E. Framework-Side Scans

Table VIII reports the pre-flight diagnostics scan of [1] §8.5 across the 16 benchmark scenarios. For each scenario at the peak-violation tick of the C_1 run, the active rule-encoder gradients were extracted and the combined FM I LICQ + FM II convexity check (`lcp.run_diagnostics`) was applied. Convexity holds structurally because every rule encoder emits affine inequalities $a^T z + b^T u + e \leq 0$ (FM II passes by construction); the LICQ check reports the rank of the combined active-gradient matrix and confirms full column rank on every scenario — the framework’s standing geometric assumption is satisfied across the benchmark.

Table IX reports the empirical §10.2 necessity scan. For each scenario, the fraction of applicable ticks at which each priority level had non-zero violation is computed from the C_1 run; a level is considered “effectively hard” iff its violation fraction is below an empirical tolerance (1%); i_{nec}^* is the largest level for which the level and every higher-priority level was effectively hard. Forced-relaxation levels are those above i_{nec}^* . On scenarios such as 11_protected_cross the geometry of the turn intersects the opposing-lane half-plane, so level L_7 γ_{r2} is forced-relaxed — a deterministic consequence of the road geometry rather than a planner deficiency. The scan ties the field behaviour back to [1]’s Definition 10.1 and identifies the scenarios where the framework operates in the necessity-required relaxation regime.

F. Summary

The empirical evidence confirms the four deployment claims

of Section I-D, and the structured framework-side validation summarised in Fig. 18 confirms each algorithmic property the deployment depends on. Priority-respecting compromise is realised on every scenario (§VII-A): the operational LCP planner concentrates rule violations at the low-priority levels (L_0 – L_3) and holds the high-priority levels (L_7 – L_{10}) effectively hard. The operational weighted-sum solve reproduces the formal cascade’s compliance pattern at the levels held effectively hard (§VII-B): the median per-tick compliance match rate is 1.000 at L_{10} and L_2 , 0.987 at L_9 , 0.981 at L_8 — the operational realisation of [1]’s Theorem 4.1 at the levels that hold; at the levels engaged in the priority-respecting compromise (L_7 , L_3 , L_1 , L_0) the median match rate is 0.664–0.839 as per-tick numerical differences toggle the compliance bit around the tolerance ϵ_ℓ . The architectural cost reduction promised by [1] is realised in wall time (§VII-C): the operational planner runs at median $1/6.22 \approx 16\%$ of the cascade’s wall time (range $1/9.39$ – $1/1.76$, $\approx 11\%$ – 57%). The framework’s standing assumptions hold (§VII-E): pre-flight diagnostics pass on every scenario (Table VIII) and the empirical §10.2 scan (Table IX) identifies the scenarios on which the deployment operates in the necessity-required relaxation regime, ascribing those rule violations to the road geometry rather than to planner suboptimality.

VIII. DISCUSSION

We close the empirical analysis with four discussion threads: what the framework buys, what it does not (limitations), how it compares with the principal prior approaches, and what could threaten the validity of our claims.

A. What the Framework Buys

The deployment described in Section IV and validated by the protocol of Section V produces four operational gains.

A structural priority guarantee. The calibrated weighted-sum planner (C_1) is provably equivalent to the lex cascade (C_4) at the active set on which $w^\dagger$ was calibrated (Theorem II.1; the v10_2 Support-Normal Equivalence). The empirical compliance match rate of Section VII (§VII-B) operationalises this guarantee at the levels held effectively hard in deployment (median match rate ≥ 0.981 at L_8 , L_9 , L_{10} collision-avoidance levels and at L_2 route adherence): the operational weighted-sum planner reproduces the formal cascade’s compliance pattern in closed loop at the priority-critical levels, not just in theory.

An order-of-magnitude reduction in online solve cost. The cascade requires $L + 1$ sequential nonlinear-program solves per tick; the calibrated weighted-sum mode requires one. The empirical ratio at our parameters (Section VII §VII-C) is larger than the structural lower bound because each cascade stage runs on a strictly tighter feasibility set than its predecessor, which raises the per-stage interior-point iteration count and weakens warm-starting. The architectural decomposition of [1] purchases this reduction with an offline calibration step (cached per Section IV-J); the trade is asymmetric in favour of online throughput.

A runtime diagnostic for active-set drift. The runtime compliance checker (§IV-K) detects the operational failure mode of weighted-sum scalarisation: when the active set drifts away from the calibration’s, the equivalence guarantee weakens silently. The checker raises a per-tick flag for every per-level mismatch. In the protocol reported here, the action policy is log-only; the same per-tick signal is the input to the cascade-fallback and Algorithm 1A/1B recalibration policies of [1] §9.6 (implemented in `lcp.deploy_tick`) that would close the loop in production.

A practitioner-facing lessons roster. Table III consolidates the eleven implementation primitives and the observer subsystem as a roster mapping each lesson to its problem, symptom, workaround, and rejected alternative. The roster is the artefact a practitioner needs to replicate the deployment on a different rule set, vehicle, or simulator; it is the contribution of the present manuscript that is independent of any specific empirical outcome.

B. Limitations

The framework and its deployment carry six explicit limitations that we make load-bearing.

Approximate equivalence around the lex active set. The exact-equivalence theorem (Theorem II.1) holds at the active set at which $w^\dagger$ was calibrated. The SLP outer loop (§IV-C) re-linearises the dynamics about the warm-start at every tick; the active set on the linearised problem may differ from the active set at the warm-start. The compliance match-rate evidence of Section VII (§VII-B) quantifies the drift empirically: the approximation is faithful at the levels held effectively hard (median match rate 1.000 at L_{10} and L_2 , 0.987 at L_9 , 0.981 at L_8) and degrades at the levels engaged in the priority-respecting compromise (median 0.664 at L_7 , 0.696 at L_3 , 0.839 at L_1 , 0.772 at L_0). The match-rate split is not monotonic in priority depth — it is structured by whether the level is held hard or actively trading off the per-tick compromise.

Cross-step coupling not natively expressible. The convex per-step template (§IV-B) expresses only single-step inequalities. Cross-step couplings (lateral jerk, longitudinal jerk, multi-step deceleration profiles) enter the OCP only through single-step proxies (§IV-G); the proxies are conservative on smooth-trajectory regimes and conservative-in-expectation on burst regimes.

State-machine rules out of scope. The 9 observer-only rules of Table I encode multi-tick state-machine semantics (mandatory-stop approach, yield-priority arbitration, intersection-blocking detection) that are not natively expressible in the convex per-step template. Extending the framework to encompass these rules would require discrete mode variables (incompatible with the convex template) or a multi-step temporal extension (a research direction not pursued here).

Polygonal disjunctions reduce to closest-face half-planes. The polygon-to-half-plane reduction (§IV-E) is exact when the warm-start lies inside the closest-face Voronoi cell of the polygon. On the Voronoi boundary the heuristic resolves ties by picking any of the equidistant faces; both choices yield a consistent half-plane normal direction so the emitted constraint

is conservative when the warm-start lies outside the polygon and permissive when inside. In either case the SLP outer loop re-linearises on the next iteration about the new warm-start, away from the boundary.

Single-vehicle, single-simulator evidence. All empirical claims in this manuscript are based on a single vehicle model (Chrysler Pacifica), a single simulator (nuPlan-mini), and a single seed. Cross-vehicle generalisation, cross-simulator generalisation, and cross-region (driving-culture) generalisation are open questions, as is multi-seed stochastic robustness; the qualitative pattern of priority-respecting compromise was the structural property of interest in this paper, so single-seed evidence is sufficient to establish it.

Necessity-forced relaxations are scenario-dependent. The empirical §10.2 scan (Table IX) identifies the scenarios on which the planner operates in the necessity-required relaxation regime: `l1_protected_cross` forces `7r2` (opposing-lane) relaxation because the geometry of the turn intersects the opposing-lane half-plane. Deployments in regions or scenarios with different road geometries will experience different necessity-forced relaxation patterns; the scan is the operator’s runtime tool for identifying which level violations are geometric inevitabilities rather than planner deficiencies.

C. Comparison with Prior Approaches

We position the deployment against three prior approaches.

Hierarchical Quadratic Programming (HQP). HQP [6], [7] treats the cascade as the operational construct, solving the $L + 1$ -stage problem per tick. HQP is the gold standard if the per-tick budget admits $L + 1$ solves; in our setting the per-tick budget does not, and the calibrated weighted-sum mode is the operational alternative. The cascade condition C_4 of our protocol is the natural HQP reference: it is precisely the lex-cascade solution on the linearised inner problem, which is what HQP would compute per tick at the same priority depth. The computational asymmetry against the calibrated weighted-sum substitution is sharp: HQP performs $L + 1$ convex sub-solves per tick (online cost $\Theta(L + 1)$), whereas the calibrated weighted-sum mode performs the lex cascade once per scenario class offline (cost amortised across $\sim 10^2$ – 10^3 ticks per cell) plus a single weighted-sum solve per tick (online cost $\Theta(1)$).

Flat-weight MPC. Flat-weight MPC tunes a single set of soft-collision-slack weights and lacks structural priority semantics. Even with carefully tuned weights, the flat-weight planner has no mechanism to guarantee that, e.g., a comfort violation will not be accepted in exchange for a safety violation; the violation vector is a single scalar trade-off. The priority-respecting compromise pattern of Section VII (§VII-A) is exactly the structural property that flat-weight tuning cannot match.

Goal programming with Sherali’s equivalent weights. Sherali [3] established the existence and computability of equivalent weights for lexicographic linear programs. The foundation paper [1] extends this to the convex setting via the support-normal characterisation. The present manuscript extends the prior tradition operationally: we add the runtime mismatch

detector (§IV-K) absent from Sherali’s framework, provide the offline calibration cache that makes Algorithm 1A tractable at deployment scale, and embed the calibration in a closed-loop autonomous-vehicle simulator with the eleven implementation primitives of Table III.

D. Threats to Validity

Three threats to the validity of the empirical claims merit explicit acknowledgement.

Scenario distribution. The 16 nuPlan-mini scenarios are a curated subset of the full nuPlan benchmark. The distribution over scenario classes is approximately balanced (turn, lane-change, follow, intersection traversal) but is not random; conclusions about the framework’s behaviour on scenarios outside this distribution are extrapolations.

Single-seed evidence. We use a single random seed for the dual-condition protocol. The qualitative structural property of priority-respecting compromise we report is invariant under seed (the relative priorities of rule violations are determined by the planner’s encoder and weight calibration, not by the simulator’s random initial condition). Multi-seed stochastic robustness is a separate empirical question not addressed by this paper.

Calibration-cache freshness. The cache is populated once per scenario class at the start of the protocol. If the scenario instances within a class drift in their active-set structure during the simulation (a foreseeable failure mode in long-horizon deployment), the calibration ages and the equivalence guarantee weakens. The runtime compliance checker (§IV-K) detects this; the recalibration policy is implemented in `lcp.deploy_tick` but its action policy is left to the operator.

IX. CONCLUSION AND FUTURE WORK

A. Summary

We have presented a closed-loop deployment of Lexicographic Constraint Programming for trajectory Model Predictive Control on the nuPlan simulator. Building on the geometric equivalence framework of [1], we have (i) released a dynamics-agnostic reference implementation that reproduces the framework’s published numerical answers to 10^{-6} precision via a 55-test regression suite covering Algorithms 0, 1A, 1B, the pre-flight FM I/FM II diagnostics, the online cascade-fallback policy, the Relaxation Decision Framework, and the upper-image reification of [1] §3; (ii) realised the calibrated weighted-sum operational mode and the lex cascade as a reference inside a two-level MPC pipeline with a Sequential Linear Programming outer loop, consolidated as a roster of eleven implementation primitives plus the observer subsystem (Table III); and (iii) demonstrated the deployment on the 16-scenario nuPlan-mini benchmark in a dual-condition protocol where the operational planner (C_1) and the formal cascade (C_4) were run on each scenario.

The empirical evidence supports the four deployment claims summarised in Section VII-F: priority-respecting compromise is realised on every scenario (the low-priority comfort and headway levels L_0 – L_3 absorb the violation mass;

the high-priority collision levels L_8 – L_{10} remain effectively held; Figs. 12–14); the operational weighted-sum solve reproduces the formal cascade’s per-tick compliance pattern at the effectively-held levels with median match rate 1.000 at L_{10} and L_2 , 0.987 at L_9 , 0.981 at L_8 — the operational realisation of [1]’s Theorem 4.1 in closed loop (Fig. 15); the architectural wall-time benefit of substituting the calibrated weighted-sum solve for the $L+1$ -stage cascade is realised at median $\rho^{(s)} = 6.22$ across the 16 scenarios (range 1.76–9.39; Table VII); and the framework’s standing assumptions (FM I LICQ + FM II convexity) hold across the benchmark (Table VIII), with the empirical §10.2 necessity scan (Table IX) identifying the scenarios on which the deployment legitimately operates in the necessity-required relaxation regime.

The present work does not address the multi-tick state-machine rules of Table I (mandatory-stop approach, yield-priority arbitration, block-the-box detection, uncontrolled-intersection negotiation, unmarked-crosswalk yield); the nine observer-only rules fall outside the convex per-step template and are not encoded by the planner. Extending the framework to encompass them is a hybrid-MPC research direction (point 3 of Section IX-B below) beyond the present scope.

B. Future Work

For future work, we are interested in extending the deployment along four directions.

An inverse problem on the equivalence region. The equivalence region $\Omega(p^*)$ of Theorem II.1 is a polyhedron in weight space whose interior contains all weights that recover the lex optimum. The inverse problem treats $\Omega(p^*)$ as a feasibility set for weight inference from expert demonstrations: given a corpus of human-driven trajectories, find the weight vector $w^\dagger$ that best explains the demonstrations subject to lying in $\Omega(p^*)$. The inverse-problem formulation reconciles the framework’s structural priorities with operator-supplied or learned weight preferences.

Online recalibration on first compliance-check mismatch. The runtime compliance checker (§IV-K) currently logs mismatches but the recalibration policy is operator-supplied. The `lcp.deploy_tick` entry point implements the [1] §9.6 cascade-fallback and Algorithm 1A/1B recalibration policies; wiring these into the production planner so the cache updates in place on the first mismatch would close the loop between the offline and online phases of the operational workflow of Fig. 4. The engineering challenge is to perform the recalibration within a few ticks; the algorithmic challenge is to handle the race condition between the recalibration and the next tick’s deployment.

Hybrid Model Predictive Control for state-machine rules. The 9 observer-only rules of Table I sit outside the convex template because their semantics require discrete-mode reasoning. A hybrid extension that augments the convex per-step OCP with discrete mode variables (mandatory-stop approach states, yield-priority modes, intersection-blocking detection) would extend the lex-dominance guarantee from the 13 MPC-controlled rules to a larger subset of the rulebook at the cost of mixed-integer programming overhead.

Cross-vehicle and multi-seed deployment. All empirical claims in this manuscript are tied to a single vehicle model (Chrysler Pacifica) and a single random seed. Cross-vehicle deployment requires re-calibration of the kinematic bicycle parameters, re-tuning of the per-rule encoder thresholds, and re-population of the calibration cache for the new platform. Multi-seed stochastic-robustness evidence would complement the single-seed qualitative pattern presented here. The framework and the deployment are vehicle-agnostic and seed-agnostic in structure; the engineering work to scale to a second platform or to a multi-seed protocol is well-scoped by the lessons roster of Table III.

REFERENCES

- [1] H. Hajieghrary, “Lexicographic constraint programming: A geometric equivalence framework for convex priority-ordered MPC,” 2026, companion foundation manuscript, submitted concurrently with the present deployment paper; provided as supplementary material. Establishes Theorem 4.1, Propositions 5.1/6.1, the failure-mode catalogue of §8, and Algorithms 1A/1B with the worked examples reproduced as Appendix B verification anchors in the present paper.
- [2] A. Censi, K. Slutsky, T. Wongpiromsarn, D. Yershov, S. Pendleton, J. Fu, and E. Frazzoli, “Liability, ethics, and culture-aware behavior specification using rulebooks,” in *International Conference on Robotics and Automation*, 2019, pp. 8536–8542.
- [3] H. D. Sherali, “Equivalent weights for lexicographic multi-objective programs: characterizations and computations,” *European Journal of Operational Research*, vol. 11, no. 4, pp. 367–379, 1982.
- [4] F. Eisenbrand, S. Niermann, and M. Skutella, “Lexicographic optimization in linear and combinatorial programs: explicit weight bounds,” *Mathematical Programming*, 2003.
- [5] A. Charnes and W. W. Cooper, *Management Models and Industrial Applications of Linear Programming*. Wiley, 1961.
- [6] O. Kanoun, F. Lamiroux, and P.-B. Wieber, “Kinematic control of redundant manipulators: generalizing the task-priority framework to inequality task,” *IEEE Transactions on Robotics*, vol. 27, no. 4, pp. 785–792, 2011.
- [7] A. Escande, N. Mansard, and P.-B. Wieber, “Hierarchical quadratic programming: fast online humanoid-robot motion generation,” *International Journal of Robotics Research*, vol. 33, no. 7, pp. 1006–1028, 2014.
- [8] K. Esterle, P. Hart, J. Bernhard, and A. Knoll, “Spatiotemporal motion planning for autonomous vehicles with trapezoidal prism corridors and Bézier curves,” in *American Control Conference*, 2020.
- [9] S. Shalev-Shwartz, S. Shammah, and A. Shashua, “On a formal model of safe and scalable self-driving cars,” *arXiv preprint arXiv:1708.06374*, 2017.
- [10] IEEE Vehicular Technology Society, “IEEE 2846: Assumptions for models in safety-related automated vehicle behavior,” IEEE Standard 2846-2022, 2022.
- [11] A. P. Wierzbicki, *The use of reference objectives in multiobjective optimization*. Springer, 1980.
- [12] P.-L. Yu, *Multiple-Criteria Decision Making: Concepts, Techniques, and Extensions*. Plenum Press, 1985.
- [13] M. Zeleny, *Multiple Criteria Decision Making*. McGraw-Hill, 1982.
- [14] K. Miettinen, *Nonlinear Multiobjective Optimization*. Kluwer, 1999.
- [15] J. Branke, K. Deb, H. Dierolf, and M. Osswald, “Finding knees in multi-objective optimization,” in *Parallel Problem Solving from Nature*, 2004, pp. 722–731.
- [16] I. Das, “On characterizing the “knee” of the Pareto curve based on normal-boundary intersection,” *Structural Optimization*, vol. 18, no. 2–3, pp. 107–115, 1999.
- [17] U. Schwesinger, M. Rufli, and R. Siegwart, “A sampling-based partial motion planning framework for system-compliant navigation along a reference path,” *Robotics and Autonomous Systems*, 2018.
- [18] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [19] D. P. Bertsekas, *Nonlinear Programming*, 2nd ed. Athena Scientific, 1999.
- [20] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

- [21] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi: a software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [22] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, “nuPlan: a closed-loop ML-based planning benchmark for autonomous vehicles,” in *Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [23] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” in *Conference on Robot Learning (CoRL)*, 2023, introduces PDM-Closed, the canonical strong rule-based baseline on nuPlan (CoRL 2023 challenge winner; CLS-R $\approx$ 92 on Val14).
- [24] R. T. Rockafellar, *Convex Analysis*. Princeton University Press, 1970.
- [25] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [26] I. Hasuo, “Responsibility-sensitive safety: an introduction with an eye to logical foundations and formalization,” *arXiv preprint arXiv:2206.03418*, 2022.

APPENDIX A

HYPERPARAMETER REFERENCE

We report the complete set of load-bearing hyperparameters used by both conditions of the dual-condition protocol (Section V). The values are the defaults of the Hydra config workspace/lexicone/planning/config/planner/two_level_mpc_planner.yaml; the only per-condition override is runtime_mode (ws for C_1 , cascade for C_4) called out in Table ?? . The common-parameter design isolates the effect of the inner-OCP strategy as the only varying factor.

We additionally report the hidden constants of the deployment that are not exposed as Hydra keys but affect the empirical results.

- *Tikhonov regulariser* $\eta = 10^{-6}$ on the L_1 violation slacks (§IV-H); ensures the reduced Hessian is positive definite without biasing the L_1 semantics to first order.
- *Slot budgets per OCP level* [16, 7, 11] for (L_S, L_L, L_C) , including the stub slot capacity for 10r5, 7r4, 3r6; total 34 epigraph slacks per step.
- *Inactive-slot offset* $-M = -10^6$ (§IV-F); large enough to make masked epigraph constraints trivially satisfied.
- *Vehicle parameters* from nuPlan’s get_pacifica_parameters() (Chrysler Pacifica): wheel base $\ell = 3.089$ m, front overhang 0.94 m, rear overhang 1.13 m, body width 2.02 m, body length $\ell + 0.94 + 1.13 = 5.16$ m, circumscribed radius $r_{\text{ego}} \approx 2.78$ m used by the collision encoder (§IV-D). All four parameters feed the kinematic-bicycle model of (8) and the inflated-circle keep-out constraint of Appendix C.

APPENDIX B

ALGORITHM PSEUDOCODE (0, 1A, 1B)

We supply here the pseudocode for Algorithm 0 (the homogeneous-cone primary formulation of [1] §9.1; the detailed pseudocode for Algorithms 1A and 1B already appears inline in Sections II-F and II-G) together with complexity bounds and the empirical verification anchors for all three.

Algorithm 0 (homogeneous-cone primary, [1] §9.1)

The homogeneous-cone primary formulation is operator-supplied-box-free: the lower bounds $\underline{w}_i, \underline{\beta}$ are positivity floors

Algorithm 3 ALG. 0 — HOMOGENEOUS-CONE PRIMARY

Require: active rule-encoder gradients $\{\nabla g_{i,j}\}$ at z_{lex}^* , performance gradient ∇J , hard inequalities $\{\nabla g_{\text{phys},p}\}$, equalities $\{\nabla h_e\}$; per-level lower bounds $\underline{w}_i > 0$; performance-coefficient lower bound $\underline{\beta} > 0$.

Ensure: unit-performance weights $\bar{w} \in \Omega(p^*)$.

- 1: Form variables $[w \in \mathbb{R}^L \mid \beta \in \mathbb{R} \mid \alpha \in \mathbb{R}^{n_{\text{bdy}}} \mid \lambda \in \mathbb{R}^{n_{\text{phys}}} \mid \mu \in \mathbb{R}^{n_{\text{eq}}} \mid r \in \mathbb{R}]$.
- 2: Add the d -dim KKT stationarity equality $\beta \nabla J + \sum_{(i,j)} \alpha_{i,j} \nabla g_{i,j} + \sum_p \lambda_p \nabla g_{\text{phys},p} + \sum_e \mu_e \nabla h_e = 0$.
- 3: Add the simplex-slice equality $\beta + \sum_i w_i = 1$.
- 4: Add Chebyshev face slacks: $w_i - \underline{w}_i \geq r$; $\beta - \underline{\beta} \geq r$; $\alpha_{i,j} \geq r$; $w_i - \alpha_{i,j} \geq r$; $\lambda_p \geq r$; $r \geq 0$.
- 5: Solve max r by linear programming, yielding $(w^\#, \beta^\#, r^\#)$.
- 6: **return** $\bar{w} := w^\# / \beta^\#$.

only and the simplex-slice equality bounds the LP by construction. The projection $\bar{w}_i = w_i^\# / \beta^\#$ recovers unit-performance WS weights in $\Omega(p^*)$.

Complexity

For fixed priority depth L and fixed weight box $\mathcal{W}$, Algorithm 1 runs in time polynomial in the size A of the cascade-active-set signature p^* . The cascade solve at step 1 is one nonlinear program at convex-QP complexity (interior-point at the inner OCP, with $O(N \cdot d)$ decision variables for horizon N and per-step state-control dimension d). The KKT-system formation at step 2 is $O(A)$. The Fourier–Motzkin elimination at step 3 produces a half-space description of $\Omega(p^*)$ with at most $|H| = O(A^2)$ inequalities (the elimination doubles the number of constraints per eliminated variable in the worst case; the cascade structure typically keeps $|H|$ much smaller in practice). The Chebyshev-centre linear program at step 4 has $L + 1$ unknowns (the weight vector w and the inscribed-ball radius r) and $|H| + 2L$ constraints (the half-space description plus the per-dimension weight-box bounds), solved by `scipy.optimize.linprog` in polynomial time in $|H|$.

Algorithm 2 adds the singular-perturbation sensitivity-constant extraction at step 2 (one reduced-KKT linear system) and the coupled-linear formation at step 3. The verification step at step 5 reruns the WS solve at $w^\dagger$ and reads the resulting per-level violations; one additional convex-QP solve.

Algorithm 3 is the same Chebyshev-LP construction as Algorithm 1A but on the homogeneous-cone simplex slice rather than the operator-supplied box; the LP has $L + 1 + n_{\text{bdy}} + n_{\text{phys}} + n_{\text{eq}} + 1$ unknowns and one more equality constraint than Algorithm 1A. Its complexity is therefore the same up to constants.

Verification anchors

All three algorithms are unit-tested in the deployment codebase against the analytical worked examples of [1] §11:

- Algorithm 1 against Example 1 ([1] §11.1, two-priority L_1 LP) reproduces $w^\dagger = (5.5, 5.5)$, $r^\dagger = 4.5$,

$\beta_{\text{at_opt}} = (1.0, 1.0)$; the equivalence region returned by `omega_half_space_description` reduces to $\{w_1 \geq 1\} \cap \{w_2 \geq 1\}$ as the paper states.

- Algorithm 2 against Example 2 ([1] §11.2, three-priority L_2 QP) reproduces the sensitivity constants $c_1 = 7$ and $\kappa_1 = (0, 0, 0.75)$; the threshold function returns $W_1(\epsilon_1 = 0.01, w_3 = 1) \approx 77.5$ and $W_1(\epsilon_1 = 0.01, w_3 = 10) = 145$; the paper’s exhibited witness $(200, 100, 10)$ achieves un-divided threshold slack $0.1 \cdot 200 - 0.75 \cdot 10 - 7 = 5.5$.
- Algorithm 3 against Example 1 produces a projected $\bar{w}$ satisfying $\bar{w}_1, \bar{w}_2 \geq 1$ (i.e., inside $\Omega(p^*)$) without an operator-supplied box.

The tests live at `workspace/lcp/tests/test_equivalence`, `test_equivalence_alg1a.py`, and `test_equivalence_alg1b.py`. The full 55-test suite (Algorithms 0/1A/1B + diagnostics + online deployment + relaxation + cache + compliance + upper image) passes in < 1 s and is the empirical evidence that the `lcp` reference implementation reproduces [1] faithfully to 10^{-6} precision.

APPENDIX C ENCODER SPECIFICATION

We document the twelve encoder modules of the deployment. Each encoder translates one observer rule (or, in the case of the combined collision encoder, two) into an affine convex inequality slot in the OCP (7). The per-step slot budget per encoder, the convex form, and the operational masking semantics are listed in Table XI. The 13 MPC-controlled rule IDs of Table I are covered by 12 encoders (the collision encoder `CollisionRule` is shared between rules `9r0` and `10r0`).

Per-step coefficient triple

Each slot of each encoder emits a per-(slot, step) coefficient triple $(a, b, e) \in \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}$ for use in the convex inequality (7): $a^T \tilde{x}_k + b^T \tilde{u}_k + e \mu \leq T_{\ell,j,k}$. The triple is computed from the encoder’s per-tick *warm-start linearisation*: for a non-linear underlying constraint $g(x_k, u_k) \leq 0$ evaluated at the SLP warm-start $(\bar{x}_k, \bar{u}_k)$, the encoder emits

$$a = \nabla_x g(\bar{x}_k, \bar{u}_k), \quad b = \nabla_u g(\bar{x}_k, \bar{u}_k), \quad e = g(\bar{x}_k, \bar{u}_k) - a^T \bar{x}_k - b^T \bar{u}_k. \quad (15)$$

The inequality $a^T \tilde{x}_k + b^T \tilde{u}_k + e \leq T_{\ell,j,k}$ is then the first-order Taylor expansion of g at the warm-start, satisfied tightly when the SLP fixed point coincides with the warm-start. For the `CollisionRule` (inflated-circle keep-out vs. agent j at position o_j with radius r_j), the underlying g is $g(x_k) = (r_j + r_{\text{ego}})^2 - (x_k - o_{j,x})^2 - (y_k - o_{j,y})^2$ and the linearised coefficients are $a_x = -2(\bar{x}_k - o_{j,x})$, $a_y = -2(\bar{y}_k - o_{j,y})$, with the remaining components zero. For the affine encoders (`SpeedLimitRule`, `LongitudinalComfortRule`, `LateralAccelerationRule`, etc.) the linearisation is exact and (a, b, e) reduce to constants per step.

The applicability mask $\mu_{\ell,j,k} \in \{0, 1\}$ is the encoder’s per-step decision on whether the slot is operational on that tick; when $\mu = 0$, the encoder sets $e \leftarrow -M$ with $M = 10^6$, rendering the constraint

trivially satisfied at $T_{\ell,j,k} = 0$ regardless of the dynamics solution. The encoder implementations are at `workspace/lexicone/planning/rule_encoder.py`; the partition test `test_rule_level_mapping.py` asserts that the encoder set matches the `MPC_CONTROLLED_IDS` declaration at module import time, providing machine-checked invariance of the count under refactoring.

APPENDIX D FRAMEWORK VERIFICATION

This appendix consolidates the framework-side evidence that the `lcp` reference implementation is a faithful realisation of the `v10_2` specification. The evidence is layered: (D.1) the worked examples of [1, §11] reproduced at 10^{-6} precision; (D.2) the seven-dimension structured validation report (Fig. 18 of the main manuscript); (D.3) the cross-reference between `v10_2` sections and `lcp/` modules.

D.1 Worked-example reproduction

The `lcp/tests/` `pytest` suite contains 55 tests; the 11 that exercise the paper’s worked examples are summarised in Table XII.

Both examples are also exercised end-to-end (diagnostics $\rightarrow$ calibration $\rightarrow$ cascade-vs-WS $\rightarrow$ compliance $\rightarrow$ §10.2 necessity) by the demo scripts `workspace/examples/lcp_demo_v10_2.py` (Example 1) and `workspace/examples/lcp_demo_example2.py` (Example 2); their captured stdout is in the supplement (§I).

D.2 Seven-dimension validation battery

The script `workspace/examples/validate_v10_2.py` (Section VII-F) exercises the seven evaluation dimensions of Fig. 18:

- 1) **Correctness** (Algorithm-level reproduction of the worked examples).
- 2) **Completeness** (every public `lcp.*` symbol imports and `is_callable`; 51/51).
- 3) **Performance** (Algorithm 1A LP wall-time at $L = 2, 3, 5, 8, 12$; 0.7–1.0 ms per solve).
- 4) **Effectiveness** (Theorem 4.1: 200/200 random weights in $\Omega(p^*) \cap [1, 10]^2$ recover the lex optimum $z = (3, 5)$).
- 5) **Utility** (Algorithm-1A vs Algorithm-0 vs flat-weight vs badly-chosen vs lopsided: only the two calibrated strategies recover lex).
- 6) **Perturbation stability** (Theorem 7.1 in the L_1 regime: 200 random multiplicative perturbations of $w^\dagger$; every in- Ω perturbation preserves the binary compliance vector $b = (1, 1)$; every out-of- Ω perturbation drops at least one compliance bit).
- 7) **Procedure 10.1 convergence** (constructed three-level problem combining one necessity-forced level, one utility-relaxed level, and one hard level: converges in two iterations to $(\delta_1, \delta_2, \delta_3) = (0, 0.5, 0.4)$).

D.3 Section-to-module map

Table XIII resolves every v10_2 section (theory and application) to its concrete implementation. The dynamics-agnostic theory of Sections 2–11 lives in `lcp/`; the bicycle-specific deployment content of Sections 12 and 14 lives in `lexicone/planning/`.

APPENDIX E REPRODUCIBILITY MANIFEST

E.1 Build chain

The end-to-end reproduction of every figure, table, and numerical claim in this manuscript runs in three phases.

a) Phase A — framework regression.: Verify the `lcp` reference implementation against [1]’s worked examples.

```
cd workspace
python -m pytest lcp/tests/          # 55 tests
python examples/lcp_demo_v10_2.py    # Example 1 end-to-end
python examples/lcp_demo_example2.py
```

The regression suite reproduces $w^\dagger = (5.5, 5.5)$, $r^\dagger = 4.5$ for Example 1; $c_1 = 7$, $\kappa_1[3] = 0.75$, witness slack 5.5 for Example 2. The two demo scripts replay the full pipeline (diagnostics $\rightarrow$ Algorithm 1A/1B $\rightarrow$ cascade-vs-WS solve $\rightarrow$ compliance check $\rightarrow$ §10.2 necessity probe) end-to-end against the paper-published numbers.

b) Phase B — closed-loop dual-condition protocol.: Re-run the 32-scenario protocol on `nuPlan-mini`.

```
cd workspace

# C1 (operational WS-L1; ~2 min/scenario, ~30 min total)
NUPLAN_EXP_ROOT=/workspace/nuplan-project/exp \
python examples/12_batch_two_level_mpc_planner.py \
  --penalty-form l1 --runtime-mode ws --seed 7 \
  --output-dir examples/outputs/manuscript/C1_instrumented

# C4 (cascade reference; ~14 min/scenario, ~4 h total)
NUPLAN_EXP_ROOT=/workspace/exp_worker_c4/exp \
python examples/12_batch_two_level_mpc_planner.py \
  --penalty-form l1 --runtime-mode cascade --seed 7 \
  --output-dir examples/outputs/manuscript/C4_cascade
```

Each batch emits one MP4 + summary PNG + per-tick `*_log.csv` per scenario, plus a per-batch `batch_summary_l1.csv`.

c) Phase C — artefact generation + LaTeX build.:

```
cd workspace
bash examples/regen_manuscript.sh    # diagnostics + necessity scans
                                     # + figures + tables to IEEE_T-IV/

cd ../IEEE_T-IV
pdflatex Main && bibtex Main \
&& pdflatex Main && pdflatex Main
```

The `regen_manuscript.sh` script invokes `scan_diagnostics.py`, `scan_necessity.py`, and `make_manuscript_artefacts.py`, which mirror the generated figures into

`IEEE_T-IV/Figures/` and the `\input{}`able tables into `IEEE_T-IV/Sections/tables/`.

E.2 Software versions

The deployment uses the Python scientific stack with the following pinned versions (the project `requirements.txt` is the authoritative source).

E.3 Hardware spec

The protocol was executed on a single workstation. Each batch runs `worker=sequential` (one scenario at a time per process) and the two batches can be parallelised across distinct `NUPLAN_EXP_ROOT` directories. Memory peak per process is dominated by the `nuPlan SimulationLog` buffer (~ 1 GB per 15-second scenario). Disk footprint per scenario is ~ 50 MB (MP4 + summary PNG + per-tick CSV); the full 32-scenario protocol fits in a few GB.

TABLE III: Lessons learned from the closed-loop LCP-MPC deployment on nuPlan. Each row consolidates one implementation primitive (or, in the final row, the observer subsystem) as a four-column entry: the operational problem it solves, the failure symptom expected if the primitive is absent, the workaround we adopted, and the alternative we considered but rejected. The roster is the central contribution of Section IV and is cross-referenced from Section VIII-A.

Primitive (§)	Problem solved	Symptom under ablation	Workaround chosen	Alternative considered
SLP outer loop (§IV-C)	Kinematic bicycle dynamics are non-convex; the convex framework assumes convexity.	Tracking error grows on high-curvature ticks; OCP becomes infeasible near aggressive turns.	Sequential linearisation about the warm-start with re-solve until trajectory residual < 0.05 m.	Full sequential quadratic programming (SQP): more expensive per outer iteration; harder to warm-start.
Rule encoders (§IV-D)	The convex framework requires per-tick affine inequalities; observer rules are described semantically.	Without encoders the OCP has no rule-driven shape and the planner reduces to a kinematic tracker.	12 encoders covering 13 MPC-controlled rule IDs; each encoder emits per-(slot, step) coefficients (a, b, e) .	One encoder per rule (12 vs. 13 cardinality through the combined collision encoder); per-rule duplication.
Polygon-to-half-plane reduction (§IV-E)	Walkway and lane polygons induce non-convex disjunctions (“avoid the union of these polygons”).	The OCP becomes a mixed-integer program with discrete polygon selectors; intractable at 10 Hz.	Closest-face heuristic keyed on the warm-start: emit one half-plane per slot, against the nearest polygon face.	Union of half-spaces (large per-slot budget); soft-constraint formulation (loses the exact-feasibility guarantee).
Applicability masking (§IV-F)	The number of applicable rules per tick varies; the OCP’s variable layout must be static for CasADi precompilation.	Dynamic slot allocation breaks warm-starting; per-tick CasADi function rebuilds add seconds of latency.	Static maximum slot budget per level; per-(level, slot, step) binary mask $\mu \in \{0, 1\}$; offset $e \leftarrow -M$ when inactive.	Dynamic OCP rebuild per tick ($10\times$ slower); single over-large fixed budget without masking (wastes slots).
Cross-step coupling (§IV-G)	Temporal-derivative rules (jerk, lateral jerk) span multiple ticks; the per-step linear template cannot express them.	Single-step proxies are conservative; the exact constraint is unenforceable.	Single-step proxies (e.g., $ a_y $ bound as a one-step lateral-jerk proxy) calibrated against the offline cascade.	Two-step or three-step template (more decision variables); approximate via terminal cost (loses the tight bound).
Ego-local-frame solving (§IV-H)	UTM coordinates at 10^5 – 10^6 m destroy IPOPT’s internal scaling; squared-distance penalties overflow.	IPOPT iteration limit reached; warm-start broken on every tick; convergence stalls.	Transform to a frame with the ego rear axle at the origin, heading along $+x$; solve; transform back.	Manual variable-scaling matrix on world-frame variables (error-prone); switch to CVXPY (slower).
L_1 Tikhonov regularisation (§IV-H)	The L_1 violation functional is non-smooth; the reduced Hessian is not strictly positive definite at the solution.	IPOPT line-search stalls; tick-to-tick warm-start drifts.	Add a $\eta \sum T_{i,j,k}^2$ regulariser with $\eta = 10^{-6}$; keeps semantics unbiased to first order.	Switch to L_2 penalty (loses exact-equivalence regime); replace IPOPT with a non-smooth solver (slower).
WS–cascade dual mode (§IV-I)	The cascade is the formal reference but $\approx 7\times$ slower than weighted sum; weighted sum needs calibration.	Single-mode deployment forces a difficult speed-vs-fidelity tradeoff.	Two modes: w_s (calibrated weighted sum) for online deployment; cascade for offline reference + fallback.	Weighted sum only (no formal reference for the comparative protocol); cascade only (cannot meet 10 Hz).
Offline calibration cache (§IV-J)	Re-running Algorithm 1 per tick is prohibitive ($\approx$ tens of seconds per scenario class).	8–15 s of latency per scenario-class entry; real-time deployment impossible.	JSON-backed cache keyed on (scenario class, penalty form, conditions across ticks); offline pre-population.	Amortised online recalibration (race conditions across ticks); per-scenario fresh calibration (no cache benefit).
Runtime compliance checker (§IV-K)	The weighted-sum solution may drift from the lex optimum when the active set changes between ticks.	Silent degradation of priority semantics; the framework’s lex guarantee is lost without notification.	Per-tick comparison $b_\epsilon(z_{ws}^*) \stackrel{?}{=} b_\epsilon^*$; mismatch is logged, optional cascade fallback.	Trust the calibration unconditionally (silently breaks); always run the cascade (defeats the calibration’s purpose).
Serialisation-tolerance contract (§IV-L)	nuPlan’s <code>SimulationLog.save_t</code> end of every simulation; pickles the planner; CasADi SwigPyObjects cannot pickle.	Pickle exception at the end of every simulation; the log is unsavable.	<code>__getstate__</code> drops CasADi attributes (and the new per-tick CSV handles); <code>__setstate__</code> rebuilds.	Skip log save (loses replayability); pickle-friendly wrapper around CasADi (large refactor).
Observer subsystem (§IV-N)	Planner-side encoders are convex proxies; the truth-of-record needs a venue-agnostic per-rule evaluator.	The planner asserts “rule X respected” but the assertion is unverified against the simulator state.	25-rule observer engine runs in parallel, evaluates each rule per tick, emits independent rule-evaluation records.	Trust the planner-side encoders (no cross-check); reuse nuPlan’s bundled metrics (loses cross-venue generality).

TABLE VII: Per-scenario wall-time of the operational C_1 and the cascade C_4 and the ratio $\rho^{(s)}$ (Definition V.4).

Scenario	T_{C_1} (s)	T_{C_4} (s)	$\rho^{(s)}$
01_following_slow_lead	154.2	1234.6	8.01
02_near_long_vehicle	161.9	1278.3	7.90
03_near_multiple_vehicles	164.8	960.2	5.83
04_changing_lane	115.4	798.9	6.92
05_changing_lane_left	100.7	799.7	7.94
06_changing_lane_right	106.7	762.4	7.15
07_starting_left_turn	162.5	1060.1	6.52
08_starting_right_turn	182.6	888.3	4.86
09_high_speed_turn	197.2	1530.6	7.76
10_low_speed_turn	330.5	909.2	2.75
11_protected_cross	237.0	1070.4	4.52
12_unprotected_cross	221.8	797.4	3.60
13_high_magnitude_speed	132.8	1247.2	9.39
14_medium_magnitude_speed	244.5	867.1	3.55
15_near_high_speed_vehicle	434.7	766.4	1.76
16_traversing_intersection	201.8	1194.7	5.92
Median	—	—	6.22

TABLE VIII: Pre-flight diagnostics ([1] §8.5) per benchmark scenario. LICQ rank reports the rank of the combined active-gradient matrix at the peak-violation tick of the C_1 run; framework applies iff FM I (LICQ full rank) and FM II (convexity) both hold.

scenario	peak_active_count	licq_rank	licq_n_columns	licq_deficit	framework_applies
01_following_slow_lead__l1	8	8	8	0	True
02_near_long_vehicle__l1	7	7	7	0	True
03_near_multiple_vehicles__l1	6	6	6	0	True
04_changing_lane__l1	8	8	8	0	True
05_changing_lane_left__l1	8	8	8	0	True
06_changing_lane_right__l1	3	3	3	0	True
07_starting_left_turn__l1	6	6	6	0	True
08_starting_right_turn__l1	3	3	3	0	True
09_high_speed_turn__l1	5	5	5	0	True
10_low_speed_turn__l1	6	6	6	0	True
11_protected_cross__l1	6	6	6	0	True
12_unprotected_cross__l1	6	6	6	0	True
13_high_magnitude_speed__l1	3	3	3	0	True
14_medium_magnitude_speed__l1	4	4	4	0	True
15_near_high_speed_vehicle__l1	7	7	7	0	True
16_traversing_intersection__l1	5	5	5	0	True

TABLE IX: Empirical §10.2 necessity scan per benchmark scenario. i_{nec}^* is the largest priority level for which level i_{nec}^* and every higher-priority level had $\leq 1\%$ of applicable ticks violating; *forced_levels* lists the levels above i_{nec}^* that exceeded the hardness tolerance.

scenario	i_star_nec	forced_levels	per_level_violation_fr
01_following_slow_lead__l1	10	0,1,2,3,7,8,9	L0=0.167;L1=0.480;L
02_near_long_vehicle__l1	10	0,1,2,3,7,8,9	L0=0.083;L1=0.429;L
03_near_multiple_vehicles__l1	10	0,1,2,3,7,9	L0=0.190;L1=0.058;L
04_changing_lane__l1	9	0,2,3,7,8	L0=0.029;L1=0.003;L
05_changing_lane_left__l1	9	0,2,3,7,8	L0=0.029;L1=0.003;L
06_changing_lane_right__l1	-1	0,2,3,10	L0=0.020;L1=0.000;L
07_starting_left_turn__l1	10	0,1,2,3,7,8,9	L0=0.151;L1=0.304;L
08_starting_right_turn__l1	-1	0,1,2,7,10	L0=0.023;L1=0.209;L
09_high_speed_turn__l1	9	0,1,2,3,7,8	L0=0.013;L1=0.203;L
10_low_speed_turn__l1	10	0,1,2,3,7,9	L0=0.010;L1=0.082;L
11_protected_cross__l1	9	0,1,2,3,7,8	L0=0.070;L1=0.101;L
12_unprotected_cross__l1	10	1,2,3,7,9	L0=0.007;L1=0.084;L
13_high_magnitude_speed__l1	4	0,1,2,3	L0=0.010;L1=0.392;L
14_medium_magnitude_speed__l1	-1	1,2,3,9,10	L0=0.003;L1=0.176;L
15_near_high_speed_vehicle__l1	10	1,2,3,7,8,9	L0=0.000;L1=0.209;L
16_traversing_intersection__l1	-1	0,1,2,3,7,10	L0=0.013;L1=0.245;L

TABLE XI: Encoder roster for the LCP-MPC deployment. Each row specifies the encoder class, the observer rule it encodes, the OCP priority level, the per-step slot budget, and the convex inequality form. The combined collision encoder (CollisionRule) covers both 9r0 (non-VRU vehicles) and 10r0 (VRUs); the three stub encoders (10r5, 7r4, 3r6) reserve OCP slot capacity for future enforcement but emit inactive constraints today.

Encoder	Rule(s)	Lvl.	Slots	Convex form
CollisionRule	9r0, 10r0	L_S	8	Per-agent inflated circle keep-out: $(x_k - o_{j,x})^2 + (y_k - o_{j,y})^2 \geq (r_j + r_{ego})^2$ linearised.
LaneCorridorRule	7r0	L_S	2	Two linear half-planes $\pm n^T(p_k - p_{ref}) \leq w_{corridor}$ around the route centre-line.
SidewalkDriveRule	7r5	L_S	4	Closest face half-plane of the nearest walkway polygon to the warm-start.
SpeedLimitRule	3r0	L_L	1	$v_k - v_{lim}(s_k) \leq 0$, with the arc-length projection.
OpposingLaneRule	7r2	L_L	2	Half-plane vs the nearest opposing-direction lane.
OneWayDirectionRule	7r3	L_L	2	Half-plane variant

TABLE XII: Worked-example regression tests for the lcp/ reference implementation. Each test reproduces a published numerical answer from [1] §11. All 55 lcp/tests/ tests pass in < 1 s.

Test	Source ref.	Reproduced quantity
<i>Example 1 (L_1 two-priority LP, [1] §11.1)</i>		
Alg. 1A on Ex. 1 box $[1, 10]^2$	§11.1	$w^\dagger = (5.5, 5.5)$, $r^\dagger = 4.5$
Alg. 1A β multipliers	§11.1	$\beta_{at_opt} = (1.0, 1.0)$
$\Omega(p^*)$ half-spaces	Eq. (15)	$\{-w_1 \leq -1\} \cap \{-w_2 \leq -1\}$
Alg. 0 projected $\bar{w}$ Lex image p^*	§9.1 Eq. (2)	$\bar{w} \in \Omega(p^*)$ $(V_1, V_2, J) = (0, 0, -11)$
<i>Example 2 (L_2 three-priority QP, [1] §11.2)</i>		
Sensitivity constants	§11.2	$c_1 = 7$, $\kappa_1[3] = 0.75$
Threshold at $w_3 = 1$	Eq. (21)	$W_1(\epsilon_1=0.01) \approx 77.5$
Threshold at $w_3 = 10$	Eq. (21)	$W_1(\epsilon_1=0.01) = 145$
Paper-witness slack at $(200, 100, 10)$	§11.2	$0.1 \cdot 200 - 0.75 \cdot 10 - 7 = 5.5$
Tight ϵ_1 scaling	Thm. 6.1	$w_1^\dagger \propto 1/\sqrt{\epsilon_1}$

TABLE XIII: Cross-reference between v10_2 sections and the lcp/ reference implementation. The lcp/ package is dynamics-agnostic; the Section 12 (application to rulebook MPC) and Section 14 (deployment lessons) content lives one layer up in workspace/lexicone/planning/ where the bicycle-specific deployment glue is.

v10_2 §	Module / function in lcp/
§2 Preliminaries	lcp.ConvexPriorityProblem, LevelSpec, AffineDynamics, BoxBounds
§3 Upper image	lcp.AchievementImage, lex_image_from_cascade, verify_lex_extreme_point
§4 Thm. 4.1 (equivalence)	lcp.algorithm_1a encodes KKT for $\Omega(p^*)$
§5 L_1 polyhedral	lcp.omega_half_space_description
§6 L_2 asymptotic	lcp.compute_l2_sensitivity_constants, l2_threshold
§7 Compliance	lcp.ComplianceChecker
§8 Failure modes	lcp.check_licq, check_convexity, run_diagnostics
§9.1 Algorithm 0	lcp.algorithm_0_homogeneous
§9.3 Algorithm 1A	lcp.algorithm_1a
§9.4 Algorithm 1B	lcp.algorithm_1b
§9.5 Cache	lcp.CalibrationCache
§9.6 Online deployment	lcp.deploy_tick, OnlineDeploymentConfig
§10 Relaxation framework	lcp.compute_necessary_relaxation_level, find_knee_depth, iterative_lex_relaxation
§11 Worked examples	workspace/lcp/tests/test_equivalence_alg{0
§12 Application	workspace/lexicone/planning/ (Section IV)
§14 Deployment lessons	workspace/lexicone/planning/ + Table III

